

DDD V1 – Section 1

Database Design Principles

Status: FROZEN

1.1 Purpose

The TODP database exists to support:

- Opportunity Discovery
- Authority Discovery
- Evidence Storage
- Recommendation Generation
- Auditability
- Provenance
- Enterprise RAG Retrieval
- Observability

The database is not merely a storage layer.

It is a traceability and evidence system.

Every recommendation, ranking, citation, source, audit result, and retrieval operation must be traceable through the database.

1.2 Database Philosophy

TODP follows the principle:

No Evidence

=

No Strong Recommendation

Therefore:

Recommendation

↓

Evidence

↓

Source

↓

Citation

↓

Timestamp

must always be recoverable.

The database is the system of record.

The LLM is never the system of record.

1.3 Database Technology Stack

Primary Database

PostgreSQL 17+

Purpose:

- Transactional Storage
 - Relationship Management
 - Audit Storage
 - Version Storage
-

Vector Extension

pgvector

Purpose:

- Embedding Storage
 - Semantic Search
 - Similarity Search
 - Hybrid Retrieval
-

Database Classification

Relational Database

+

Vector Database

implemented inside PostgreSQL.

1.4 Core Architectural Principles

Principle 1

Trace Everything

Every entity must be traceable.

Examples:

- Opportunity
 - Source
 - Citation
 - Recommendation
 - Audit Result
-

Principle 2

Audit Continuously

Every recommendation must be auditable.

Every ranking must be reproducible.

Principle 3

Preserve Provenance

All evidence must preserve:

- Origin
 - Source
 - Citation
 - Timestamp
-

Principle 4

Version Everything

Changes must never overwrite history.

Historical versions must remain accessible.

Principle 5

Reproduce Everything

A recommendation generated today must be reproducible later using:

- Same Data
 - Same Evidence
 - Same Ranking Formula
 - Same Version
-

1.5 Database Design Goals

The database must support:

Goal 1

Authority Discovery

Goal 2

Opportunity Discovery

Goal 3

Evidence Retrieval

Goal 4

Recommendation Generation

Goal 5

Recommendation Audit

Goal 6

Hallucination Detection

Goal 7

Observability

Goal 8

Enterprise RAG Retrieval

1.6 Domain-Driven Structure

The database is divided into domains.

User Domain

Stores:

- Users
 - Preferences
 - Consents
 - Locations
-

Authority Domain

Stores:

- Authorities
 - Authority Hierarchies
 - Authority Sources
-

Opportunity Domain

Stores:

- Events
 - Scholarships
 - Programs
 - Courses
-

Evidence Domain

Stores:

- Evidence
 - Evidence Versions
-

Source Domain

Stores:

- Sources
- Source Metadata

Citation Domain

Stores:

- Citations
-

Recommendation Domain

Stores:

- Recommendations
 - Ranking Results
-

Audit Domain

Stores:

- Audit Logs
 - Hallucination Audits
-

Retrieval Domain

Stores:

- Retrieval Runs
 - Retrieval Results
-

1.7 Immutable Audit Principle

Audit records are immutable.

Once created:

Audit Record

Cannot Be Modified

Allowed:

Create New Audit Record

Not Allowed:

Update Historical Audit Record

Purpose:

- Legal Defensibility
 - Traceability
 - Reproducibility
-

1.8 Evidence-First Principle

The database must support:

Evidence

↓

Recommendation

Never:

Recommendation

↓

Evidence

Evidence must exist before recommendation generation.

1.9 Recommendation Traceability Principle

Every recommendation must be linked to:

- Source
- Citation
- Evidence Chain
- Retrieval Timestamp
- Ranking Run
- Audit Result

through database relationships.

1.10 Ranking Storage Principle

The database must store:

- Money Cost

- Time Cost
- Distance Cost
- Ranking Score

for every recommendation.

The database must never store only final rankings.

The factors that created the ranking must also be stored.

1.11 Retrieval Intelligence Principle

The database must support:

- BM25 Retrieval
- Vector Retrieval
- Metadata Filtering
- Hybrid Retrieval
- Reranking

for Enterprise RAG workflows.

1.12 Authority Hierarchy Principle

Authority structures must never be hardcoded.

Authority relationships must be stored.

Examples:

```
College
↓
University
↓
Regulator
↓
Ministry

Municipality
↓
District
↓
State
↓
National
```


must be represented as database relationships.

1.13 Discovery Hierarchy Principle

Discovery hierarchy:

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

must be represented through stored relationships and metadata.

1.14 Scalability Principle

The database design must support migration from:

PostgreSQL + pgvector

to future architectures without changing the business model.

Examples:

PostgreSQL + Qdrant

PostgreSQL + Distributed Retrieval

Multi-Region Architecture

1.15 Section 1 Acceptance Criteria

Section 1 is complete only if the database supports:

✓ Traceability

✓ Auditability

- ✓ Provenance
 - ✓ Versioning
 - ✓ Recommendation Reproducibility
 - ✓ Authority Discovery
 - ✓ Enterprise RAG Retrieval
 - ✓ Hallucination Auditing
 - ✓ Ranking Storage
 - ✓ Evidence-First Architecture
-

Section 1 Status

DDD V1 Section 1

STATUS: FROZEN

DDD V1 – Section 2

Entity Relationship Architecture

2.1 Purpose

This section defines the logical relationship structure between all major TODP domains.

The objective is to ensure:

- Traceability
- Auditability
- Provenance
- Recommendation Reproducibility
- Enterprise RAG Retrieval Support

Every recommendation must be traceable through database relationships back to:

```
Recommendation
↓
Evidence
↓
Citation
↓
Source
↓
Authority
```

2.2 Entity Relationship Philosophy

TODP follows:

Nothing Exists In Isolation

Every major entity must be connected through explicit relationships.

Examples:

```
Recommendation
→ Evidence

Evidence
→ Citation

Citation
```

→ Source

Source

→ Authority

Authority

→ Hierarchy

2.3 High Level Domain Model

User Domain

↓

Authority Domain

↓

Opportunity Domain

↓

Evidence Domain

↓

Citation Domain

↓

Recommendation Domain

↓

Audit Domain

↓

Retrieval Domain

2.4 Core Entity Relationship Diagram

Users

|

— User Preferences

— User Consents

— User Locations

|

— Recommendations

|

— Ranking Results

— Audit Results

— Evidence Chains

Authorities

|

— Authority Hierarchy

— Authority Sources

— Authority Versions

Opportunities

|

— Events

— Scholarships

— Programs

— Courses

└─ Providers

Evidence

└─ Citations
└─ Sources
└─ Evidence Versions
└─ Retrieval Results

2.5 User Domain Relationships

Primary Entity:

users

Relationships:

users
└─ user_preferences
└─ user_consent
└─ user_locations
└─ calendar_entries
└─ registrations

Relationship Type:

One User
→ Many Preferences

One User
→ Many Consents

One User
→ Many Locations

One User
→ Many Registrations

One User
→ Many Calendar Entries

2.6 Authority Domain Relationships

Primary Entity:

authorities

Relationships:

```
authorities
├── authority_hierarchy
├── authority_sources
├── authority_versions
```

Example:

```
Government of India
├── Ministry
│   ├── Department
│   │   └── Program
```

Example:

```
University
├── College
│   └── Course
```

2.7 Opportunity Domain Relationships

Primary Entity:

```
opportunities
```

Relationships:

```
opportunities
├── opportunity_categories
├── opportunity_locations
├── opportunity_providers
├── evidence
├── recommendations
```

Examples:

```
Event
```

```
Scholarship
```

```
Course
```

Government Program

Workshop

All opportunities must belong to at least one provider.

2.8 Evidence Domain Relationships

Primary Entity:

evidence

Relationships:

evidence
├── citations
├── sources
├── evidence_versions
└── retrieval_results

Rule:

One Evidence Record

May Reference

Multiple Citations

2.9 Citation Domain Relationships

Primary Entity:

citations

Relationships:

citations
└── sources

Rule:

Citation

Must Reference

Exactly One Source

2.10 Source Domain Relationships

Primary Entity:

sources

Relationships:

sources

- |— authority_sources
- |— evidence
- |— citations

Examples:

Government Website

University Website

NGO Website

Official Program Page

2.11 Recommendation Domain Relationships

Primary Entity:

recommendations

Relationships:

recommendations

- |— ranking_runs
- |— evidence
- |— audits
- |— opportunities

Rule:

Every Recommendation

Must Reference

Evidence

2.12 Ranking Domain Relationships

Primary Entity:

ranking_runs

Relationships:

ranking_runs
├── ranking_factors
├── recommendations
└── audits

Stored Values:

Money Cost

Time Cost

Distance Cost

Ranking Score

2.13 Audit Domain Relationships

Primary Entity:

audit_logs

Relationships:

audit_logs
├── recommendations
├── ranking_runs
├── retrieval_runs
└── evidence

Rule:

Audit Records

Are Immutable

2.14 Retrieval Domain Relationships

Primary Entity:

retrieval_runs

Relationships:

```
retrieval_runs
├── retrieval_queries
├── retrieval_results
├── evidence
└── recommendations
```

Supports:

BM25

Vector Search

Hybrid Retrieval

Metadata Filtering

Reranking

2.15 Enterprise RAG Relationship Layer

```
Documents
├── Chunks
├── Embeddings
├── Retrieval Results
└── Evidence
```

Relationship Flow:

```
Document
↓
Chunk
↓
Embedding
↓
Retrieval Result
↓
Evidence
```

↓
Recommendation

2.16 Authority Discovery Relationship Layer

Authority
↓
Parent Authority
↓
Hierarchy
↓
Discovery Scope

Examples:

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

2.17 Recommendation Traceability Graph

Recommendation
↓
Ranking Run
↓
Evidence
↓
Citation
↓
Source
↓
Authority

Every recommendation must be traceable through this chain.

2.18 Cardinality Rules

Examples:

One User
→ Many Recommendations

One Authority
→ Many Opportunities

One Opportunity
→ Many Evidence Records

One Evidence Record
→ Many Citations

One Recommendation
→ Many Audit Records

One Retrieval Run
→ Many Retrieval Results

2.19 Relationship Integrity Rules

Foreign key constraints must enforce:

- Referential Integrity
- Provenance Integrity
- Audit Integrity
- Recommendation Integrity

Orphan records are not allowed.

2.20 Section 2 Acceptance Criteria

Section 2 is complete only if:

- ✓ Every recommendation is traceable
- ✓ Every evidence object is linked
- ✓ Every citation is linked
- ✓ Every source is linked
- ✓ Every authority is linked
- ✓ Every retrieval result is linked

- ✓ Every audit record is linked
 - ✓ Enterprise RAG relationships are supported
 - ✓ Authority hierarchy relationships are supported
 - ✓ Recommendation reproducibility is supported
-

DDD V1 – Section 3

User Domain Architecture

3.1 Purpose

The User Domain manages:

- User Identity
- User Preferences
- User Consents
- User Locations
- User Discovery Context
- User Registrations
- User Calendar Associations

The purpose of the User Domain is not profiling.

The purpose is to provide context required for:

- Discovery
- Ranking
- Registration Assistance
- Calendar Assistance

while preserving:

- Traceability
 - Auditability
 - Reproducibility
-

3.2 User Domain Philosophy

TODP does not rank users.

TODP does not score users.

TODP does not classify users using subjective metrics.

Users are context providers.

Examples:

Location

Institution

Discovery Scope

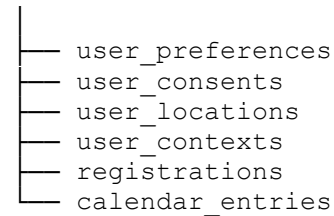
Calendar Availability

Consent Settings

These are operational attributes only.

3.3 User Domain Entity Diagram

users



3.4 Users Table

Primary Entity:

users

Purpose:

Store user account information.

Primary Key

user_id

Core Fields

user_id

email
display_name
account_status
created_at
updated_at
version

Rules

One User
Must Have
One Unique User ID

3.5 User Preferences Table

Primary Entity:

user_preferences

Purpose:

Store configurable user settings.

Examples

Preferred Discovery Radius
Preferred Language
Preferred Notification Settings
Calendar Preferences

Relationship

users

1

↓

Many

user_preferences

Core Fields

preference_id

user_id

preference_key

preference_value

created_at

version

3.6 User Consents Table

Primary Entity:

user_consents

Purpose:

Store explicit user permissions.

Supported Consents

Calendar Access

Location Access

Registration Autofill

Account Linking

Core Fields

consent_id

user_id
consent_type
consent_status
granted_at
revoked_at
version

Rule

No Consent

=

Feature Disabled

3.7 User Locations Table

Primary Entity:

user_locations

Purpose:

Store user location context.

Used for:

- Discovery Hierarchy
 - Ranking
 - Distance Calculation
-

Core Fields

location_id
user_id
country
state

district
city
postal_code
latitude
longitude
created_at
version

Rule

Location history must remain available.

Historical records must not be overwritten.

3.8 User Context Table

Primary Entity:

user_contexts

Purpose:

Store current discovery context.

Examples:

Institution
College
University
Organization
Department

Core Fields

context_id

user_id
context_type
context_value
source_id
citation_id
created_at
version

Purpose

Supports discovery hierarchy:

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

3.9 Registration Relationships

Relationship

users

↓

registrations

Purpose

Store:

Scholarship Registrations

Program Registrations

Event Registrations

Course Registrations

Cardinality

One User

↓

Many Registrations

3.10 Calendar Relationships

Relationship

users

↓

calendar_entries

Purpose

Store:

Scheduled Events

Scheduled Courses

Scheduled Programs

Cardinality

One User

↓

Many Calendar Entries

3.11 User Discovery Relationship Model

User

↓

Location

↓
Institution
↓
Discovery Scope
↓
Opportunity Discovery

Purpose

Support discovery prioritization.

Examples:

Institution Opportunities

Local Opportunities

District Opportunities

State Opportunities

National Opportunities

International Opportunities

3.12 User Recommendation Relationship Model

User
↓
Discovery Context
↓
Retrieved Opportunities
↓
Ranking
↓
Recommendations

The user is never directly ranked.

Only opportunities are ranked.

3.13 User Audit Relationship Model

User
↓
Recommendation
↓

Audit Record

Purpose:

Allow future auditability of:

- Discovery
- Recommendation
- Registration Assistance

3.14 User Domain Versioning Strategy

Every user-related entity must contain:

version

created_at

updated_at

Historical records must remain recoverable.

Examples:

Location History

Consent History

Preference History

Institution History

3.15 User Domain Security Rules

Sensitive fields:

Email

Location

Calendar Information

Registration Information

must be protected.

Audit logs must capture:

Consent Changes

Location Changes

Context Changes

3.16 User Domain Retrieval Rules

User information must never be directly used as evidence.

User information is:

Context

not:

Evidence

3.17 User Domain Traceability Rules

Every user-related action must be traceable through:

User ID

Timestamp

Version

Audit Record

3.18 User Domain Integrity Rules

Foreign keys must enforce:

User Integrity

Consent Integrity

Location Integrity

Context Integrity

Orphan records are not allowed.

3.19 User Domain Scalability Rules

The User Domain must support:

Single User

Thousands of Users

Millions of Users

without redesigning relationships.

3.20 Section 3 Acceptance Criteria

Section 3 is complete only if:

- ✓ User identity is supported
- ✓ User preferences are supported
- ✓ User consents are supported
- ✓ User locations are supported
- ✓ User context is supported
- ✓ Registration relationships are supported
- ✓ Calendar relationships are supported
- ✓ Versioning is supported
- ✓ Auditability is supported

✓ Traceability is supported

DDD V1 – Section 4

Authority Domain Architecture

4.1 Purpose

The Authority Domain is responsible for storing and managing:

- Authorities
- Authority Hierarchies
- Authority Sources
- Authority Definitions
- Authority Relationships
- Authority Versions

The Authority Domain enables:

- Authority Discovery
 - Definition Discovery
 - Hierarchical Discovery
 - Provenance Tracking
 - Recommendation Traceability
-

4.2 Authority Domain Philosophy

TODP never hardcodes authorities.

TODP discovers authorities from evidence.

The system stores:

Authority

Definition

Hierarchy

Source

Citation

Timestamp

Version

The system does not assume:

Who Defines What

The system retrieves evidence and stores relationships.

4.3 Authority Domain Entity Diagram

authorities

- authority_hierarchy
- authority_sources
- authority_definitions
- authority_versions
- authority_aliases
- authority_jurisdictions

4.4 Authorities Table

Primary Entity:

authorities

Purpose:

Store authority records.

Examples:

Government of India

UGC

AICTE

University

College
Municipality
District Administration
UN Organization

Primary Key

authority_id

Core Fields

authority_id
authority_name
authority_type
authority_status
created_at
updated_at
version

4.5 Authority Types

Supported Types

Government
Ministry
Department
Regulator
University
College
Organization
Company
NGO

International Body

Program Provider

Authority types are configurable.

They are not hardcoded.

4.6 Authority Hierarchy Table

Primary Entity

`authority_hierarchy`

Purpose

Store parent-child relationships.

Core Fields

`hierarchy_id`

`parent_authority_id`

`child_authority_id`

`relationship_type`

`source_id`

`citation_id`

`created_at`

`version`

Example

Government of India

↓

Ministry of Education

↓

UGC

↓

University
↓
College

Example

Municipality
↓
District
↓
State
↓
National

4.7 Authority Definitions Table

Primary Entity

authority_definitions

Purpose

Store discovered definitions.

Core Fields

definition_id

authority_id

definition_text

source_id

citation_id

retrieval_timestamp

version

Rule

Definitions must always reference:

Source

Citation

Timestamp

4.8 Authority Sources Table

Primary Entity

authority_sources

Purpose

Store source mappings.

Core Fields

authority_source_id

authority_id

source_id

source_type

retrieval_timestamp

version

Examples

Government Website

University Website

Official Documentation

Official PDF

Official Program Page

4.9 Authority Versions Table

Primary Entity

authority_versions

Purpose

Preserve historical authority records.

Core Fields

authority_version_id

authority_id

version_number

effective_date

change_reason

created_at

Rule

Historical versions cannot be overwritten.

4.10 Authority Aliases Table

Primary Entity

authority_aliases

Purpose

Store alternative names.

Examples

Government of India

GOI
University Grants Commission

UGC

Core Fields

alias_id

authority_id

alias_name

source_id

created_at

4.11 Authority Jurisdictions Table

Primary Entity

authority_jurisdictions

Purpose

Store geographical scope.

Examples

Institution

Local

District

State

National

International

Core Fields

jurisdiction_id

authority_id

jurisdiction_type

jurisdiction_name
source_id
citation_id
version

4.12 Discovery Hierarchy Architecture

TODP uses:

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

Authority relationships must support this hierarchy.

4.13 Authority Discovery Workflow

Source Discovery
↓
Authority Extraction
↓
Definition Extraction
↓
Hierarchy Discovery
↓
Source Validation
↓
Citation Validation
↓
Version Storage
↓
Audit

4.14 Authority Traceability Model

Every authority must be traceable through:

Authority
↓
Definition
↓
Citation
↓
Source
↓
Timestamp

4.15 Authority Recommendation Model

Authority data supports:

Opportunity Discovery

Scholarship Discovery

Program Discovery

Course Discovery

Event Discovery

Authority data does not directly generate recommendations.

Authority data provides context.

4.16 Authority Audit Model

Every authority relationship must support:

Source Audit

Citation Audit

Hierarchy Audit

Version Audit

Audit records must remain immutable.

4.17 Authority Retrieval Model

Supports:

BM25

Vector Search

Metadata Filtering

Hybrid Retrieval

Reranking

Authority records must be retrievable through Enterprise RAG workflows.

4.18 Authority Integrity Rules

Foreign key constraints must enforce:

Authority Integrity

Hierarchy Integrity

Source Integrity

Citation Integrity

Version Integrity

Orphan records are not allowed.

4.19 Authority Scalability Rules

The Authority Domain must support:

Thousands of Authorities

Millions of Relationships

Historical Versions

Global Jurisdictions

without redesigning the schema.

4.20 Section 4 Acceptance Criteria

Section 4 is complete only if:

- ✓ Authority discovery is supported
 - ✓ Authority definitions are supported
 - ✓ Authority hierarchies are supported
 - ✓ Authority versions are supported
 - ✓ Authority aliases are supported
 - ✓ Authority jurisdictions are supported
 - ✓ Traceability is supported
 - ✓ Auditability is supported
 - ✓ Provenance is supported
 - ✓ Enterprise RAG retrieval is supported
-

DDD V1 – Section 5

Opportunity Domain Architecture

5.1 Purpose

The Opportunity Domain is the core business domain of TODP.

The purpose of this domain is to store and manage:

- Opportunities
- Events
- Scholarships
- Government Programs
- Courses
- Workshops
- Opportunity Providers
- Opportunity Locations

The Opportunity Domain serves as the primary source for:

- Discovery
- Ranking
- Recommendation Generation
- Registration Assistance
- Calendar Assistance

5.2 Opportunity Domain Philosophy

TODP does not create opportunities.

TODP discovers opportunities from evidence.

The system stores:

Opportunity

Provider

Location

Evidence

Citation

Source

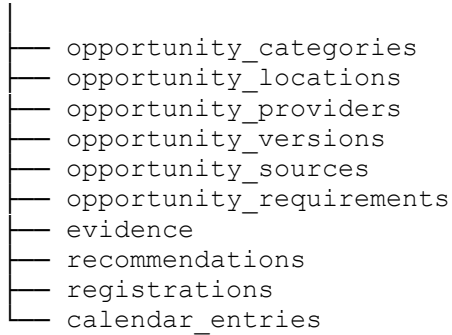
Timestamp

Version

All opportunities must be traceable.

5.3 Opportunity Domain Entity Diagram

opportunities



5.4 Opportunities Table

Primary Entity

opportunities

Purpose

Store discoverable opportunities.

Primary Key

opportunity_id

Core Fields

opportunity_id

title
description
opportunity_type
provider_id
location_id
status
created_at
updated_at
version

Supported Opportunity Types

Scholarship
Event
Workshop
Government Program
Course
Training Program
Community Initiative

5.5 Opportunity Categories Table

Primary Entity

opportunity_categories

Purpose

Store category mappings.

Examples

Education
Technology
Engineering
Medicine
Law
Business
Research

Core Fields

category_id
opportunity_id
category_name
created_at
version

5.6 Opportunity Locations Table

Primary Entity

opportunity_locations

Purpose

Store location information.

Core Fields

location_id
opportunity_id
country
state
district

city
postal_code
latitude
longitude
online_flag
created_at
version

Rule

Online opportunities must be supported.

5.7 Opportunity Providers Table

Primary Entity

opportunity_providers

Purpose

Store provider information.

Examples

University
Government Department
NGO
Company
International Organization
Training Institute

Core Fields

provider_id
provider_name
authority_id
source_id
citation_id
created_at
version

Rule

Every opportunity must have at least one provider.

5.8 Opportunity Sources Table

Primary Entity

opportunity_sources

Purpose

Store source mappings.

Examples

Official Website
Official Program Page
Official Event Page
Official PDF
Official Documentation

Core Fields

opportunity_source_id

opportunity_id
source_id
retrieval_timestamp
version

5.9 Opportunity Requirements Table

Primary Entity

opportunity_requirements

Purpose

Store eligibility and participation requirements.

Examples

Eligibility

Age Limits

Academic Requirements

Registration Requirements

Core Fields

requirement_id
opportunity_id
requirement_text
source_id
citation_id
version

5.10 Opportunity Versions Table

Primary Entity

`opportunity_versions`

Purpose

Preserve historical records.

Core Fields

`opportunity_version_id`

`opportunity_id`

`version_number`

`effective_date`

`change_reason`

`created_at`

Rule

Historical opportunity versions cannot be overwritten.

5.11 Discovery Hierarchy Model

TODP discovers opportunities using:

Institution

↓

Local

↓

District

↓

State

↓

National

↓

International

Purpose

Reduce:

- Money Cost
- Time Cost
- Distance Cost

before expanding discovery scope.

5.12 Opportunity Ranking Model

Every opportunity participates in ranking.

Ranking factors:

Money Cost

Time Cost

Distance Cost

Stored Values

Money Cost

Time Cost

Distance Cost

Ranking Score

5.13 Opportunity Recommendation Model

Relationship

Opportunity

↓

Evidence

↓

Ranking

↓

Recommendation

Rule

No Evidence

=

No Strong Recommendation

5.14 Opportunity Registration Model

Relationship

Opportunity
↓
Registration

Examples

Scholarship Registration

Program Registration

Course Registration

Event Registration

5.15 Opportunity Calendar Model

Relationship

Opportunity
↓
Calendar Entry

Examples

Event Schedule

Course Schedule

Program Schedule

5.16 Opportunity Traceability Model

Every opportunity must be traceable through:

Opportunity
↓
Provider
↓
Citation
↓
Source
↓
Timestamp

5.17 Opportunity Audit Model

Every opportunity must support:

Source Audit

Citation Audit

Ranking Audit

Recommendation Audit

Audit records remain immutable.

5.18 Opportunity Retrieval Model

Supports:

BM25

Vector Search

Metadata Filtering

Hybrid Retrieval

Reranking

Opportunity records must be retrievable through Enterprise RAG workflows.

5.19 Opportunity Integrity Rules

Foreign key constraints must enforce:

Opportunity Integrity

Provider Integrity

Location Integrity

Source Integrity

Citation Integrity

Version Integrity

Orphan records are not allowed.

5.20 Section 5 Acceptance Criteria

Section 5 is complete only if:

- ✓ Opportunity discovery is supported
- ✓ Opportunity categories are supported
- ✓ Opportunity locations are supported
- ✓ Opportunity providers are supported
- ✓ Opportunity requirements are supported
- ✓ Opportunity versions are supported
- ✓ Traceability is supported
- ✓ Auditability is supported
- ✓ Provenance is supported

✓ Enterprise RAG retrieval is supported

DDD V1 – Section 6

Evidence Domain Architecture

6.1 Purpose

The Evidence Domain is the foundation of TODP.

Every recommendation, ranking, definition, authority relationship, opportunity, and retrieval result must ultimately be backed by evidence.

The purpose of the Evidence Domain is to store:

- Evidence Records
- Evidence Relationships
- Evidence Versions
- Evidence Metadata
- Evidence Chains

while preserving:

- Traceability
 - Auditability
 - Provenance
 - Reproducibility
-

6.2 Evidence Domain Philosophy

TODP follows:

No Evidence

=

No Strong Recommendation

Every recommendation must be traceable through:

Recommendation

↓

Evidence

↓

Citation

↓

Source

↓

Authority

Evidence is the system of record.

LLM output is not evidence.

6.3 Evidence Domain Entity Diagram

```
evidence
├── evidence_versions
├── evidence_relationships
├── evidence_metadata
├── citations
├── sources
├── retrieval_results
└── recommendations
```

6.4 Evidence Table

Primary Entity

evidence

Purpose

Store evidence records.

Primary Key

evidence_id

Core Fields

evidence_id
evidence_type
evidence_text
source_id
citation_id
retrieval_timestamp
status
created_at
version

Supported Evidence Types

Authority Evidence
Definition Evidence
Opportunity Evidence
Provider Evidence
Requirement Evidence
Ranking Evidence
Retrieval Evidence

6.5 Evidence Metadata Table

Primary Entity

evidence_metadata

Purpose

Store evidence attributes.

Core Fields

metadata_id
evidence_id
metadata_key
metadata_value
created_at

Examples

Document Type
Jurisdiction
Provider
Program Type
Source Type

6.6 Evidence Relationships Table

Primary Entity

evidence_relationships

Purpose

Store relationships between evidence records.

Core Fields

relationship_id
parent_evidence_id
child_evidence_id
relationship_type
created_at
version

Examples

Authority Definition

↓

Authority Source

Opportunity

↓

Opportunity Requirement

Recommendation

↓

Ranking Evidence

6.7 Evidence Versions Table

Primary Entity

evidence_versions

Purpose

Store historical evidence versions.

Core Fields

evidence_version_id

evidence_id

version_number

effective_date

change_reason

created_at

Rule

Historical evidence must never be overwritten.

6.8 Evidence Source Relationship

Relationship

Evidence
↓
Source

Rule

Every evidence record must reference:

Source
Citation
Timestamp

Evidence without source references is invalid.

6.9 Evidence Citation Relationship

Relationship

Evidence
↓
Citation

Rule

Every evidence record must reference at least one citation.

6.10 Evidence Retrieval Relationship

Relationship

Retrieval Result
↓
Evidence

Purpose

Convert retrieved information into auditable evidence.

6.11 Evidence Recommendation Relationship

Relationship

Evidence
↓
Recommendation

Rule

Recommendations cannot exist without evidence.

6.12 Evidence Audit Model

Every evidence record must support:

Source Audit
Citation Audit
Version Audit
Retrieval Audit

Audit records must remain immutable.

6.13 Evidence Provenance Model

Every evidence record must preserve:

Origin

Source

Citation

Timestamp

Version

Provenance must never be removed.

6.14 Evidence Chain Architecture

TODP stores evidence chains.

Example:

Recommendation

↓

Evidence

↓

Citation

↓

Source

↓

Authority

Purpose

Support:

- Recommendation Audit
 - Ranking Audit
 - Hallucination Audit
-

6.15 Evidence Discovery Scope

Evidence may originate from:

Institution

Local

District

State

National

International

Discovery scope must be stored as metadata.

6.16 Evidence Integrity Rules

Foreign key constraints must enforce:

Evidence Integrity

Source Integrity

Citation Integrity

Version Integrity

Relationship Integrity

Orphan records are not allowed.

6.17 Evidence Retrieval Rules

Supports:

BM25

Vector Search

Metadata Filtering

Hybrid Retrieval

Reranking

Evidence records must be retrievable through Enterprise RAG workflows.

6.18 Evidence Scalability Rules

The Evidence Domain must support:

Millions of Evidence Records

Millions of Citations

Millions of Relationships

Historical Versions

without redesigning the schema.

6.19 Evidence Lifecycle

Discovery

↓

Retrieval

↓

Validation

↓

Evidence Creation

↓

Recommendation

↓

Audit

↓

Versioning

Evidence is never deleted.

Evidence may be:

Active

Superseded

Deprecated

while remaining recoverable.

6.20 Section 6 Acceptance Criteria

Section 6 is complete only if:

- ✓ Evidence storage is supported
 - ✓ Evidence relationships are supported
 - ✓ Evidence versions are supported
 - ✓ Provenance is supported
 - ✓ Auditability is supported
 - ✓ Traceability is supported
 - ✓ Recommendation linkage is supported
 - ✓ Citation linkage is supported
 - ✓ Source linkage is supported
 - ✓ Enterprise RAG retrieval is supported
-

DDD V1 – Section 7

Source Domain Architecture

7.1 Purpose

The Source Domain is responsible for storing and managing:

- Sources
- Source Metadata
- Source Health
- Source Versions
- Source Ownership
- Source Discovery Information

The Source Domain serves as the foundation for:

- Provenance
- Traceability
- Citation Validation
- Evidence Validation
- Recommendation Validation

Every citation must reference a source.

Every evidence record must reference a source.

Every recommendation must ultimately trace back to a source.

7.2 Source Domain Philosophy

TODP follows:

No Source
=
No Evidence

and therefore:

No Evidence
=
No Strong Recommendation

The system never trusts:

- Generated Content
- Unverified Claims
- Unsupported Statements

without a source.

7.3 Source Domain Entity Diagram

```
sources
├── source_versions
├── source_health
├── source_metadata
├── authority_sources
├── opportunity_sources
├── citations
└── evidence
```

7.4 Sources Table

Primary Entity

sources

Purpose

Store source records.

Primary Key

source_id

Core Fields

source_id

source_name

source_type

source_url

source_status

retrieval_timestamp

created_at

updated_at

version

Examples

Government Website

University Website

Official PDF

Official Documentation

Program Portal

Event Portal

Scholarship Portal

7.5 Source Types

Supported Source Types

Government Website

Government PDF

University Website

University PDF

Official Program Page

Official Event Page

Official Documentation

NGO Website

Company Website

International Organization Website

Source types must remain configurable.

7.6 Source Metadata Table

Primary Entity

source_metadata

Purpose

Store additional source attributes.

Core Fields

metadata_id

source_id

metadata_key

metadata_value

created_at

Examples

Jurisdiction

Language

Organization Type

Provider Type

Country

State

7.7 Source Health Table

Primary Entity

source_health

Purpose

Track source availability and reliability.

Core Fields

health_id

source_id

availability_status

last_checked

response_time

health_score

created_at

Examples

Active

Inactive

Deprecated

Unavailable

7.8 Source Versions Table

Primary Entity

source_versions

Purpose

Store historical source information.

Core Fields

source_version_id

source_id

version_number

effective_date

change_reason

created_at

Rule

Historical source records cannot be overwritten.

7.9 Source Ownership Model

Relationship

Source

↓

Authority

Examples

Government Website

↓

Government Authority

University Website
↓
University Authority

Program Website
↓
Program Provider

Purpose

Support provenance validation.

7.10 Source Citation Relationship

Relationship

Source
↓
Citation

Rule

One source may support:

Many Citations

Every citation must reference:

Exactly One Source

7.11 Source Evidence Relationship

Relationship

Source
↓
Evidence

Rule

One source may generate:

Many Evidence Records

Every evidence record must reference:

At Least One Source

7.12 Source Discovery Workflow

Source Discovery

↓

Source Validation

↓

Metadata Extraction

↓

Health Check

↓

Version Storage

↓

Audit

Purpose

Ensure source quality before evidence creation.

7.13 Source Validation Model

Every source must support:

Reachability Validation

Ownership Validation

Citation Validation

Evidence Validation

Invalid sources cannot generate evidence.

7.14 Source Provenance Model

Every source must preserve:

Origin

Owner

URL

Timestamp

Version

Source provenance must never be removed.

7.15 Source Audit Model

Every source must support:

Availability Audit

Ownership Audit

Citation Audit

Version Audit

Audit records remain immutable.

7.16 Source Discovery Hierarchy

Sources may belong to:

Institution

Local

District

State

National

International

Discovery scope must be stored.

7.17 Source Retrieval Model

Supports:

BM25

Vector Search

Metadata Filtering

Hybrid Retrieval

Reranking

Sources must be retrievable through Enterprise RAG workflows.

7.18 Source Integrity Rules

Foreign key constraints must enforce:

Source Integrity

Citation Integrity

Evidence Integrity

Version Integrity

Ownership Integrity

Orphan records are not allowed.

7.19 Source Scalability Rules

The Source Domain must support:

Millions of Sources

Millions of Citations

Millions of Evidence Records

Historical Versions

without redesigning the schema.

7.20 Section 7 Acceptance Criteria

Section 7 is complete only if:

- ✓ Source storage is supported
 - ✓ Source metadata is supported
 - ✓ Source health monitoring is supported
 - ✓ Source ownership tracking is supported
 - ✓ Source versions are supported
 - ✓ Citation linkage is supported
 - ✓ Evidence linkage is supported
 - ✓ Auditability is supported
 - ✓ Provenance is supported
 - ✓ Enterprise RAG retrieval is supported
-

DDD V1 – Section 8

Citation Domain Architecture

8.1 Purpose

The Citation Domain is responsible for storing and managing:

- Citations
- Citation References
- Citation Mappings
- Citation Versions
- Citation Validation Records

The Citation Domain serves as the bridge between:

```
Source
↓
Citation
↓
Evidence
↓
Recommendation
```

Every evidence record must reference citations.

Every recommendation must ultimately trace back to citations.

8.2 Citation Domain Philosophy

TODP follows:

```
No Citation
=
No Verifiable Evidence
```

and therefore:

```
No Verifiable Evidence
=
No Strong Recommendation
```

Citations are verification artifacts.

They provide traceability between evidence and sources.

8.3 Citation Domain Entity Diagram

```
citations
├── citation_mappings
├── citation_versions
├── citation_validations
├── evidence
├── sources
└── recommendations
```

8.4 Citations Table

Primary Entity

citations

Purpose

Store citation records.

Primary Key

citation_id

Core Fields

citation_id

source_id

citation_title

citation_location

citation_text

retrieval_timestamp

citation_status

created_at

version

Examples

Government Notification

University Policy

Official Scholarship PDF

Program Guidelines

Official Event Notice

8.5 Citation Mapping Table

Primary Entity

citation_mappings

Purpose

Map citations to evidence and recommendations.

Core Fields

mapping_id

citation_id

entity_type

entity_id

created_at

version

Examples

Citation

↓

Evidence

Citation
↓
Authority Definition

Citation
↓
Opportunity Requirement

8.6 Citation Validation Table

Primary Entity

`citation_validations`

Purpose

Store citation validation results.

Core Fields

`validation_id`
`citation_id`
`validation_status`
`validation_reason`
`validated_at`
`version`

Supported Status

Valid
Invalid
Deprecated
Pending Review

8.7 Citation Versions Table

Primary Entity

`citation_versions`

Purpose

Store historical citation records.

Core Fields

`citation_version_id`

`citation_id`

`version_number`

`effective_date`

`change_reason`

`created_at`

Rule

Historical citation versions must never be overwritten.

8.8 Citation Source Relationship

Relationship

Citation

↓

Source

Rule

Every citation must reference:

Exactly One Source

A citation without a source is invalid.

8.9 Citation Evidence Relationship

Relationship

Citation
↓
Evidence

Rule

One citation may support:

Many Evidence Records

Every evidence record must reference:

At Least One Citation

8.10 Citation Recommendation Relationship

Relationship

Citation
↓
Recommendation

Purpose

Enable complete recommendation traceability.

Traceability Chain

Recommendation
↓

Evidence

↓

Citation

↓

Source

8.11 Citation Discovery Workflow

Source Retrieval

↓

Citation Extraction

↓

Citation Validation

↓

Citation Storage

↓

Evidence Creation

↓

Recommendation Generation

Purpose

Ensure recommendations are built on traceable citations.

8.12 Citation Provenance Model

Every citation must preserve:

Source

Location

Timestamp

Version

Citation provenance must never be removed.

8.13 Citation Audit Model

Every citation must support:

Source Audit
Citation Audit
Version Audit
Evidence Audit

Audit records remain immutable.

8.14 Citation Retrieval Model

Supports:

BM25
Vector Search
Metadata Filtering
Hybrid Retrieval
Reranking

Citations must be retrievable through Enterprise RAG workflows.

8.15 Citation Verification Model

Verification Questions

Does the citation exist?
Is the citation reachable?
Does the citation support the evidence?
Does the citation support the recommendation?

Only verified citations may participate in recommendation generation.

8.16 Citation Lifecycle

Discovery
↓
Extraction
↓
Validation
↓
Storage
↓
Evidence Mapping
↓
Recommendation Mapping
↓
Audit
↓
Versioning

Citations are never deleted.

Supported States

Active

Superseded

Deprecated

while remaining recoverable.

8.17 Citation Integrity Rules

Foreign key constraints must enforce:

Citation Integrity

Source Integrity

Evidence Integrity

Version Integrity

Validation Integrity

Orphan records are not allowed.

8.18 Citation Scalability Rules

The Citation Domain must support:

Millions of Citations

Millions of Evidence Links

Millions of Recommendation Links

Historical Versions

without redesigning the schema.

8.19 Citation Governance Rules

The system must continuously answer:

Can every citation be traced?

Can every citation be verified?

Can every citation be audited?

Can every citation be reproduced?

These rules are mandatory.

8.20 Section 8 Acceptance Criteria

Section 8 is complete only if:

- ✓ Citation storage is supported
- ✓ Citation validation is supported
- ✓ Citation mappings are supported

- ✓ Citation versions are supported
 - ✓ Source linkage is supported
 - ✓ Evidence linkage is supported
 - ✓ Recommendation linkage is supported
 - ✓ Traceability is supported
 - ✓ Auditability is supported
 - ✓ Enterprise RAG retrieval is supported
-

DDD V1 – Section 9

Recommendation Domain Architecture

9.1 Purpose

The Recommendation Domain is responsible for storing and managing:

- Recommendations
- Recommendation Versions
- Recommendation Explanations
- Recommendation Evidence Chains
- Recommendation Rankings
- Recommendation Audit References

The Recommendation Domain serves as the final output layer of TODP.

All recommendations must be:

- Traceable
- Auditable
- Reproducible

- Explainable

9.2 Recommendation Domain Philosophy

TODP follows:

No Evidence
=
No Strong Recommendation

and

No Ranking
=
No Recommendation

Every recommendation must be supported by:

Evidence

Citation

Source

Ranking Calculation

Audit Record

The system never generates unsupported recommendations.

9.3 Recommendation Domain Entity Diagram

```
recommendations
├── recommendation_versions
├── recommendation_explanations
├── recommendation_evidence
├── ranking_runs
├── audit_logs
└── retrieval_runs
```

9.4 Recommendations Table

Primary Entity

recommendations

Purpose

Store recommendation records.

Primary Key

recommendation_id

Core Fields

recommendation_id

opportunity_id

ranking_run_id

recommendation_type

recommendation_status

generated_at

created_at

version

Examples

Scholarship Recommendation

Course Recommendation

Program Recommendation

Event Recommendation

Workshop Recommendation

9.5 Recommendation Types

Supported Types

Discovery Recommendation

Ranking Recommendation

Create Similar Recommendation

Create New Recommendation

Registration Recommendation

Calendar Recommendation

Recommendation types remain configurable.

9.6 Recommendation Evidence Table

Primary Entity

recommendation_evidence

Purpose

Store evidence mappings.

Core Fields

recommendation_evidence_id

recommendation_id

evidence_id

citation_id

source_id

created_at

version

Rule

Every recommendation must reference evidence.

9.7 Recommendation Explanations Table

Primary Entity

`recommendation_explanations`

Purpose

Store recommendation explanations.

Core Fields

`explanation_id`

`recommendation_id`

`explanation_text`

`generated_timestamp`

`version`

Rule

Explanations are derived from evidence.

Explanations are not evidence.

9.8 Recommendation Versions Table

Primary Entity

`recommendation_versions`

Purpose

Store historical recommendation records.

Core Fields

recommendation_version_id

recommendation_id

version_number

effective_date

change_reason

created_at

Rule

Historical recommendations must never be overwritten.

9.9 Recommendation Ranking Relationship

Relationship

Recommendation

↓

Ranking Run

Rule

Every recommendation must reference:

Exactly One Ranking Run

9.10 Recommendation Evidence Relationship

Relationship

Recommendation

↓

Evidence

Rule

Every recommendation must reference:

At Least One Evidence Record

9.11 Recommendation Citation Relationship

Relationship

Recommendation
↓
Citation

Purpose

Enable recommendation traceability.

Traceability Chain

Recommendation
↓
Evidence
↓
Citation
↓
Source

9.12 Recommendation Source Relationship

Relationship

Recommendation
↓
Source

Purpose

Support provenance verification.

Every recommendation must be traceable to:

One Or More Sources

9.13 Recommendation Audit Relationship

Relationship

Recommendation

↓

Audit Log

Purpose

Support:

Recommendation Audit

Ranking Audit

Evidence Audit

Hallucination Audit

9.14 Recommendation Retrieval Relationship

Relationship

Recommendation

↓

Retrieval Run

Purpose

Allow reproducibility.

Reproducibility Chain

Recommendation

↓
Retrieval Run
↓
Evidence
↓
Citation
↓
Source

9.15 Recommendation Explanation Model

Every recommendation explanation must answer:

Why was this recommended?

What evidence supports it?

What ranking factors were used?

What sources were used?

When was it retrieved?

9.16 Recommendation Lifecycle

Discovery
↓
Retrieval
↓
Evidence Creation
↓
Ranking
↓
Recommendation
↓
Audit
↓
Versioning

Recommendations are never deleted.

Supported States

Active

Superseded

Deprecated

while remaining recoverable.

9.17 Recommendation Governance Rules

The system must continuously answer:

Can every recommendation be traced?

Can every recommendation be reproduced?

Can every recommendation be audited?

Can every recommendation be explained?

These rules are mandatory.

9.18 Recommendation Integrity Rules

Foreign key constraints must enforce:

Recommendation Integrity

Evidence Integrity

Citation Integrity

Source Integrity

Ranking Integrity

Audit Integrity

Orphan records are not allowed.

9.19 Recommendation Scalability Rules

The Recommendation Domain must support:

Millions of Recommendations

Millions of Evidence Links

Millions of Ranking Records

Historical Versions

without redesigning the schema.

9.20 Section 9 Acceptance Criteria

Section 9 is complete only if:

- ✓ Recommendation storage is supported
 - ✓ Recommendation evidence mapping is supported
 - ✓ Recommendation explanations are supported
 - ✓ Recommendation versions are supported
 - ✓ Ranking linkage is supported
 - ✓ Citation linkage is supported
 - ✓ Source linkage is supported
 - ✓ Auditability is supported
 - ✓ Traceability is supported
 - ✓ Reproducibility is supported
-

DDD V1 – Section 10

Ranking Domain Architecture

10.1 Purpose

The Ranking Domain is responsible for storing and managing:

- Ranking Runs
- Ranking Factors
- Ranking Scores
- Ranking Versions
- Ranking Explanations
- Ranking Audit References

The Ranking Domain determines the ordering of opportunities before recommendations are generated.

TODP does not recommend first and justify later.

TODP ranks first, audits second, recommends third.

10.2 Ranking Domain Philosophy

TODP V1 follows:

Recommendation
Requires
Ranking

and

Ranking
Requires
Evidence

and

No Evidence
=
No Strong Recommendation

The ranking engine must be:

- Deterministic
- Reproducible
- Auditable
- Explainable

10.3 Ranking Domain Entity Diagram

```
ranking_runs
├── ranking_factors
├── ranking_scores
├── ranking_versions
├── recommendations
└── audit_logs
```

10.4 Ranking Runs Table

Primary Entity

`ranking_runs`

Purpose

Store ranking executions.

Primary Key

`ranking_run_id`

Core Fields

`ranking_run_id`

`query_id`

`user_context_id`

`ranking_formula_version`

`generated_at`

created_at

version

Rule

Every recommendation must reference exactly one ranking run.

10.5 Ranking Factors Table

Primary Entity

ranking_factors

Purpose

Store factor-level calculations.

Core Fields

factor_id

ranking_run_id

factor_name

factor_value

calculation_timestamp

version

Supported Factors

Money Cost

Time Cost

Distance Cost

10.6 Ranking Scores Table

Primary Entity

ranking_scores

Purpose

Store final ranking calculations.

Core Fields

score_id

ranking_run_id

opportunity_id

money_cost

time_cost

distance_cost

final_score

created_at

version

Rule

All factor values must be stored.

The system must never store only the final score.

10.7 Ranking Versions Table

Primary Entity

ranking_versions

Purpose

Store ranking formula history.

Core Fields

ranking_version_id

formula_name

formula_version

effective_date

change_reason

created_at

Rule

Historical ranking formulas must remain recoverable.

10.8 Ranking Formula Architecture

V1 Formula

Total Cost Score

=

Money Cost

+

Time Cost

+

Distance Cost

Lowest score ranks highest.

Examples

Lower Cost

=

Higher Rank

Higher Cost
=
Lower Rank

10.9 Money Cost Model

Purpose

Measure monetary expenditure.

Examples

Program Fee

Registration Fee

Subscription Fee

Tool Cost

Service Cost

Stored Fields

money_cost

currency

retrieval_timestamp

10.10 Time Cost Model

Purpose

Measure time expenditure.

Examples

Travel Time

Completion Time

Waiting Time

Access Time

Stored Fields

time_cost

time_unit

retrieval_timestamp

10.11 Distance Cost Model

Purpose

Measure physical distance.

Examples

Institution Distance

Local Distance

District Distance

State Distance

Stored Fields

distance_cost

distance_unit

retrieval_timestamp

10.12 Online Opportunity Rule

For online opportunities:

Distance Cost = 0

Travel Time = 0

Remaining factors:

Money Cost

Access Time

10.13 Ranking Workflow

Opportunity Discovery

↓

Evidence Validation

↓

Cost Extraction

↓

Factor Calculation

↓

Ranking Score

↓

Audit

↓

Recommendation

No ranking may bypass evidence validation.

10.14 Ranking Traceability Model

Every ranking must be traceable through:

Ranking Score

↓

Ranking Factors

↓

Evidence

↓

Citation

↓

Source

Purpose

Support complete ranking reproducibility.

10.15 Ranking Explanation Model

Every ranking must answer:

Why was this ranked first?

What costs were used?

What evidence was used?

What sources were used?

When was the ranking generated?

10.16 Ranking Audit Model

Every ranking must support:

Ranking Audit

Evidence Audit

Citation Audit

Source Audit

Recommendation Audit

Audit records remain immutable.

10.17 Ranking Reproducibility Model

The system must support:

Same Query

Same Evidence

Same Formula

Same Version



Same Ranking Result

Purpose

Ensure deterministic behavior.

10.18 Ranking Integrity Rules

Foreign key constraints must enforce:

Ranking Integrity

Factor Integrity

Evidence Integrity

Recommendation Integrity

Audit Integrity

Orphan records are not allowed.

10.19 Ranking Scalability Rules

The Ranking Domain must support:

Millions of Ranking Runs

Millions of Ranking Scores

Historical Formula Versions

Historical Ranking Results

without redesigning the schema.

10.20 Section 10 Acceptance Criteria

Section 10 is complete only if:

- ✓ Ranking runs are supported
 - ✓ Ranking factors are supported
 - ✓ Ranking scores are supported
 - ✓ Ranking versions are supported
 - ✓ Cost-based ranking is supported
 - ✓ Traceability is supported
 - ✓ Auditability is supported
 - ✓ Reproducibility is supported
 - ✓ Recommendation linkage is supported
 - ✓ Evidence linkage is supported
-

DDD V1 – Section 11

Audit Domain Architecture

11.1 Purpose

The Audit Domain is responsible for storing and managing:

- Audit Records
- Audit Results
- Audit Events
- Audit Versions
- Audit Failures
- Audit Evidence

The Audit Domain exists to verify:

- Recommendations
- Rankings
- Sources
- Citations
- Evidence
- Retrieval Operations
- Authority Relationships

The Audit Domain is a first-class system component.

11.2 Audit Domain Philosophy

TODP follows:

Trace Everything

Audit Continuously

The platform must continuously answer:

Can every recommendation be traced?

Can every ranking be reproduced?

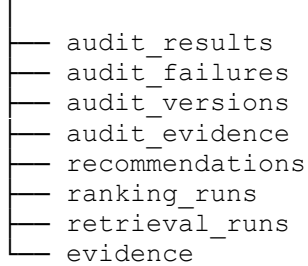
Can every source be audited?

Can every citation be verified?

Can every retrieval be reproduced?

11.3 Audit Domain Entity Diagram

audit_logs



11.4 Audit Logs Table

Primary Entity

audit_logs

Purpose

Store audit executions.

Primary Key

audit_id

Core Fields

audit_id

audit_type

entity_type

entity_id

audit_status

audit_timestamp

created_at

version

11.5 Audit Types

Supported Audit Types

Recommendation Audit

Ranking Audit

Source Audit

Citation Audit

Evidence Audit

Retrieval Audit

Authority Audit

Version Audit

Audit types remain configurable.

11.6 Audit Results Table

Primary Entity

audit_results

Purpose

Store audit outcomes.

Core Fields

result_id

audit_id

result_status
result_summary
created_at
version

Supported Status

Passed
Failed
Warning
Pending

11.7 Audit Failures Table

Primary Entity

audit_failures

Purpose

Store audit violations.

Core Fields

failure_id
audit_id
failure_type
failure_description
created_at
version

Examples

Missing Source

Missing Citation

Broken Evidence Chain

Ranking Mismatch

Invalid Authority Relationship

11.8 Audit Evidence Table

Primary Entity

`audit_evidence`

Purpose

Store evidence supporting audit conclusions.

Core Fields

`audit_evidence_id`

`audit_id`

`evidence_id`

`citation_id`

`source_id`

`created_at`

Rule

Every audit result must be evidence-backed.

11.9 Audit Versions Table

Primary Entity

`audit_versions`

Purpose

Store audit history.

Core Fields

audit_version_id

audit_id

version_number

effective_date

change_reason

created_at

Rule

Historical audit records must never be overwritten.

11.10 Recommendation Audit Model

Relationship

Recommendation

↓

Audit

Purpose

Verify:

Traceability

Evidence Coverage

Citation Coverage

Source Coverage

11.11 Ranking Audit Model

Relationship

Ranking Run
↓
Audit

Purpose

Verify:

Money Cost

Time Cost

Distance Cost

Ranking Formula

11.12 Citation Audit Model

Relationship

Citation
↓
Audit

Purpose

Verify:

Citation Exists

Citation Reachable

Citation Supports Evidence

11.13 Source Audit Model

Relationship

Source

↓
Audit

Purpose

Verify:

Source Reachable

Source Valid

Source Ownership

11.14 Evidence Audit Model

Relationship

Evidence

↓
Audit

Purpose

Verify:

Evidence Exists

Evidence Linked

Evidence Traceable

11.15 Retrieval Audit Model

Relationship

Retrieval Run

↓
Audit

Purpose

Verify:

Retrieval Completeness

Citation Coverage

Evidence Coverage

Result Reproducibility

11.16 Audit Workflow

Entity

↓

Audit Trigger

↓

Validation

↓

Evidence Collection

↓

Audit Result

↓

Storage

↓

Reporting

11.17 Audit Traceability Chain

Every audit must be traceable through:

Audit

↓

Evidence

↓

Citation

↓

Source

Purpose

Support complete audit reproducibility.

11.18 Audit Governance Rules

The system must continuously answer:

Can every audit be reproduced?

Can every audit be explained?

Can every audit be verified?

Can every audit be traced?

These rules are mandatory.

11.19 Audit Integrity Rules

Foreign key constraints must enforce:

Audit Integrity

Evidence Integrity

Citation Integrity

Source Integrity

Version Integrity

Orphan records are not allowed.

11.20 Section 11 Acceptance Criteria

Section 11 is complete only if:

- ✓ Audit logs are supported
- ✓ Audit results are supported
- ✓ Audit failures are supported
- ✓ Audit evidence is supported
- ✓ Audit versions are supported

- ✓ Recommendation audits are supported
 - ✓ Ranking audits are supported
 - ✓ Citation audits are supported
 - ✓ Source audits are supported
 - ✓ Retrieval audits are supported
-

DDD V1 – Section 12

Hallucination Audit Domain Architecture

12.1 Purpose

The Hallucination Audit Domain is responsible for detecting, recording, auditing and monitoring unsupported outputs generated by the system.

The purpose of this domain is to verify:

- Recommendations
- Rankings
- Explanations
- Summaries
- Comparisons
- Generated Plans
- Retrieved Results

against available evidence.

The Hallucination Audit Domain exists to ensure that TODP remains:

- Evidence First
- Auditable
- Traceable
- Reproducible

12.2 Hallucination Audit Philosophy

TODP follows:

No Evidence

=

No Strong Recommendation

and

AI Is Not A Source

The system must never trust:

Generated Text

Generated Explanation

Generated Summary

Generated Comparison

without evidence validation.

12.3 Hallucination Audit Entity Diagram

```
hallucination_checks
├── unsupported_claims
├── hallucination_results
├── hallucination_versions
├── hallucination_evidence
├── recommendations
├── rankings
├── retrieval_runs
└── audit_logs
```

12.4 Hallucination Checks Table

Primary Entity

hallucination_checks

Purpose

Store hallucination detection executions.

Primary Key

hallucination_check_id

Core Fields

hallucination_check_id

entity_type

entity_id

check_timestamp

check_status

created_at

version

12.5 Unsupported Claims Table

Primary Entity

unsupported_claims

Purpose

Store unsupported outputs.

Core Fields

claim_id

hallucination_check_id

claim_text

claim_type

severity

created_at

version

Examples

Missing Citation

Missing Source

Unsupported Recommendation

Unsupported Ranking

Unsupported Definition

Unsupported Explanation

12.6 Hallucination Results Table

Primary Entity

hallucination_results

Purpose

Store hallucination outcomes.

Core Fields

result_id

hallucination_check_id

result_status

result_summary

created_at

version

Supported Status

- Passed
 - Failed
 - Warning
 - Pending Review
-

12.7 Hallucination Evidence Table

Primary Entity

hallucination_evidence

Purpose

Store evidence used during hallucination validation.

Core Fields

- hallucination_evidence_id
 - hallucination_check_id
 - evidence_id
 - citation_id
 - source_id
 - created_at
-

Rule

Every hallucination result must reference supporting evidence.

12.8 Hallucination Versions Table

Primary Entity

hallucination_versions

Purpose

Store historical hallucination records.

Core Fields

hallucination_version_id

hallucination_check_id

version_number

effective_date

change_reason

created_at

Rule

Historical hallucination records must never be overwritten.

12.9 Recommendation Hallucination Model

Relationship

Recommendation

↓

Hallucination Audit

Purpose

Verify:

Recommendation Evidence

Recommendation Citation

Recommendation Source

Recommendation Traceability

12.10 Ranking Hallucination Model

Relationship

Ranking
↓
Hallucination Audit

Purpose

Verify:

Ranking Inputs
Ranking Factors
Ranking Evidence
Ranking Reproducibility

12.11 Explanation Hallucination Model

Relationship

Explanation
↓
Hallucination Audit

Purpose

Verify:

Explanation Claims
Evidence References
Citation Coverage
Source Coverage

12.12 Retrieval Hallucination Model

Relationship

Retrieval Run
↓
Hallucination Audit

Purpose

Verify:

Retrieved Evidence
Retrieved Citations
Retrieved Sources
Retrieved Recommendations

12.13 Hallucination Detection Workflow

Output Generated
↓
Claim Extraction
↓
Evidence Validation
↓
Citation Validation
↓
Source Validation
↓
Hallucination Detection
↓
Audit Storage

12.14 Hallucination Traceability Chain

Every hallucination result must be traceable through:

Claim
↓
Evidence
↓
Citation
↓
Source

Purpose

Support complete hallucination reproducibility.

12.15 Hallucination Severity Model

Supported Levels

Low

Medium

High

Critical

Examples

Missing Citation → Medium

Missing Source → High

Unsupported Recommendation → Critical

Unsupported Authority Definition → Critical

12.16 Hallucination Governance Rules

The system must continuously answer:

Can every claim be verified?

Can every recommendation be validated?

Can every explanation be traced?

Can every ranking be reproduced?

These rules are mandatory.

12.17 Hallucination Prevention Rules

The system must prevent:

Invented Sources

Invented Citations

Invented Authorities

Invented Metrics

Invented Statistics

Invented Recommendations

12.18 Hallucination Integrity Rules

Foreign key constraints must enforce:

Hallucination Integrity

Evidence Integrity

Citation Integrity

Source Integrity

Audit Integrity

Orphan records are not allowed.

12.19 Hallucination Scalability Rules

The Hallucination Domain must support:

Millions of Checks

Millions of Claims

Millions of Audit Records

Historical Versions

without redesigning the schema.

12.20 Section 12 Acceptance Criteria

Section 12 is complete only if:

- ✓ Hallucination checks are supported
 - ✓ Unsupported claims are supported
 - ✓ Hallucination evidence is supported
 - ✓ Hallucination versions are supported
 - ✓ Recommendation validation is supported
 - ✓ Ranking validation is supported
 - ✓ Explanation validation is supported
 - ✓ Traceability is supported
 - ✓ Auditability is supported
 - ✓ Reproducibility is supported
-

DDD V1 – Section 13

Retrieval Domain Architecture

13.1 Purpose

The Retrieval Domain is responsible for storing and managing:

- Retrieval Runs
- Retrieval Queries
- Retrieval Results
- Retrieval Versions
- Retrieval Metadata
- Retrieval Audit References

The Retrieval Domain is the foundation of TODP's Enterprise RAG architecture.

Its purpose is to retrieve:

- Authorities
- Definitions
- Opportunities
- Programs
- Scholarships
- Courses
- Events
- Tools
- Resources

from evidence-backed sources.

13.2 Retrieval Domain Philosophy

TODP follows:

Retrieve First

Validate Second

Rank Third

Recommend Fourth

and

No Evidence

=

No Strong Recommendation

Retrieval is not recommendation.

Retrieval produces candidate evidence.

Recommendation happens later.

13.3 Retrieval Domain Entity Diagram

retrieval_runs

- retrieval_queries
- retrieval_results
- retrieval_metadata
- retrieval_versions
- evidence
- citations
- sources
- recommendations

13.4 Retrieval Runs Table

Primary Entity

retrieval_runs

Purpose

Store retrieval executions.

Primary Key

retrieval_run_id

Core Fields

retrieval_run_id
query_id
retrieval_type
retrieval_timestamp
retrieval_status
created_at
version

Examples

Scholarship Search
Authority Search
Program Search
Course Search
Event Search

13.5 Retrieval Queries Table

Primary Entity

retrieval_queries

Purpose

Store search requests.

Core Fields

query_id
query_text
query_type
user_context_id
created_at

version

Examples

Scholarships in Jharkhand

AI Workshop

Engineering Program

Research Grant

13.6 Retrieval Results Table

Primary Entity

retrieval_results

Purpose

Store retrieval outputs.

Core Fields

result_id

retrieval_run_id

entity_type

entity_id

retrieval_score

created_at

version

Rule

Retrieval results are candidates.

They are not recommendations.

13.7 Retrieval Metadata Table

Primary Entity

retrieval_metadata

Purpose

Store retrieval attributes.

Core Fields

metadata_id

retrieval_run_id

metadata_key

metadata_value

created_at

Examples

Jurisdiction

State

Provider

Category

Source Type

13.8 Retrieval Versions Table

Primary Entity

retrieval_versions

Purpose

Store retrieval history.

Core Fields

retrieval_version_id

retrieval_run_id

version_number

effective_date

change_reason

created_at

Rule

Historical retrieval records must never be overwritten.

13.9 BM25 Retrieval Model

Supported Retrieval Method

BM25

Purpose

Support:

Keyword Search

Exact Match Search

Authority Search

Stored Fields

bm25_score

query_id

retrieval_run_id

13.10 Vector Retrieval Model

Supported Retrieval Method

Vector Search

Purpose

Support:

Semantic Search

Similarity Search

Context Search

Stored Fields

embedding_id

vector_score

retrieval_run_id

13.11 Metadata Filtering Model

Supported Retrieval Method

Metadata Filtering

Purpose

Support:

Institution

Local

District

State

National

International

Discovery hierarchy.

Stored Fields

filter_key

filter_value

retrieval_run_id

13.12 Hybrid Retrieval Model

TODP supports:

BM25

+

Vector Search

Purpose

Combine:

Keyword Retrieval

Semantic Retrieval

Stored Fields

hybrid_score

bm25_score

vector_score

13.13 Reranking Model

Purpose

Improve retrieval quality.

Workflow

Retrieval Results
↓
Reranking
↓
Candidate Results

Stored Fields

rerank_score

retrieval_run_id

version

13.14 Retrieval Discovery Workflow

Query
↓
Metadata Filter
↓
BM25 Retrieval
↓
Vector Retrieval
↓
Hybrid Retrieval
↓
Reranking
↓
Evidence Extraction

13.15 Retrieval Evidence Relationship

Relationship

Retrieval Result
↓
Evidence

Rule

Every recommendation must eventually terminate in evidence.

13.16 Retrieval Citation Relationship

Relationship

Retrieval Result
↓
Citation

Purpose

Support:

- Traceability
 - Provenance
 - Validation
-

13.17 Retrieval Source Relationship

Relationship

Retrieval Result
↓
Source

Purpose

Support:

Source Validation

Ownership Validation

Recommendation Validation

13.18 Retrieval Audit Model

Every retrieval run must support:

Retrieval Audit

Evidence Audit

Citation Audit

Source Audit

Audit records remain immutable.

13.19 Retrieval Reproducibility Model

The system must support:

Same Query

Same Metadata Filters

Same Evidence

Same Retrieval Version

↓

Same Retrieval Result

Purpose

Ensure deterministic retrieval behavior.

13.20 Section 13 Acceptance Criteria

Section 13 is complete only if:

- ✓ Retrieval runs are supported
- ✓ Retrieval queries are supported
- ✓ Retrieval results are supported
- ✓ BM25 retrieval is supported

- ✓ Vector retrieval is supported
 - ✓ Metadata filtering is supported
 - ✓ Hybrid retrieval is supported
 - ✓ Reranking is supported
 - ✓ Auditability is supported
 - ✓ Reproducibility is supported
-

DDD V1 – Section 14

Document & Chunk Domain Architecture

14.1 Purpose

The Document & Chunk Domain is responsible for storing and managing:

- Documents
- Document Versions
- Document Metadata
- Chunks
- Chunk Metadata
- Chunk Relationships

The Document & Chunk Domain serves as the ingestion foundation of TODP's Enterprise RAG architecture.

Everything retrieved by the system must originate from stored documents and chunks.

14.2 Document Domain Philosophy

TODP follows:

Document
↓
Chunk
↓
Retrieval
↓
Evidence
↓
Recommendation

and

No Document

=

No Retrieval

Documents are the origin of knowledge.

Chunks are the retrieval units.

14.3 Document & Chunk Entity Diagram

```
documents
├── document_versions
├── document_metadata
├── chunks
├── chunk_metadata
├── chunk_relationships
├── embeddings
├── retrieval_results
└── evidence
```

14.4 Documents Table

Primary Entity

documents

Purpose

Store source documents.

Primary Key

document_id

Core Fields

document_id

source_id

document_title

document_type

document_status

ingestion_timestamp

created_at

version

Examples

Government PDF

Scholarship PDF

University Notification

Program Guidelines

Official Documentation

Event Brochure

14.5 Document Types

Supported Types

PDF

HTML

DOCX

TXT

Markdown

Document types remain configurable.

14.6 Document Metadata Table

Primary Entity

`document_metadata`

Purpose

Store document attributes.

Core Fields

`metadata_id`

`document_id`

`metadata_key`

`metadata_value`

`created_at`

Examples

Language

Country

State

Provider

Authority

Category

14.7 Document Versions Table

Primary Entity

`document_versions`

Purpose

Store historical document records.

Core Fields

`document_version_id`

`document_id`

`version_number`

`effective_date`

`change_reason`

`created_at`

Rule

Historical document versions must never be overwritten.

14.8 Chunks Table

Primary Entity

`chunks`

Purpose

Store retrieval units.

Primary Key

chunk_id

Core Fields

chunk_id

document_id

chunk_text

chunk_index

chunk_status

created_at

version

Rule

Every chunk must belong to exactly one document.

14.9 Chunk Metadata Table

Primary Entity

chunk_metadata

Purpose

Store chunk attributes.

Core Fields

metadata_id

chunk_id

metadata_key

metadata_value

created_at

Examples

Page Number

Section Name

Heading

Document Type

Authority

14.10 Chunk Relationships Table

Primary Entity

chunk_relationships

Purpose

Store chunk-to-chunk relationships.

Core Fields

relationship_id

parent_chunk_id

child_chunk_id

relationship_type

created_at

version

Examples

Previous Chunk

Next Chunk

Reference Chunk

Supporting Chunk

14.11 Chunking Architecture

Supported Strategies

Fixed Size Chunking

Recursive Chunking

Semantic Chunking

Hierarchical Chunking

V1 Default Strategy

Recursive Chunking

14.12 Document Ingestion Workflow

Document
↓
Parsing
↓
Metadata Extraction
↓
Chunk Creation
↓
Embedding Generation
↓
Storage
↓
Retrieval

14.13 Chunk Retrieval Workflow

Query
↓
Chunk Retrieval
↓
Chunk Ranking
↓
Evidence Extraction
↓
Recommendation Pipeline

14.14 Document Evidence Relationship

Relationship

Document

↓

Chunk

↓

Evidence

Rule

Every evidence record must eventually trace back to a document.

14.15 Chunk Evidence Relationship

Relationship

Chunk

↓

Evidence

Purpose

Support:

- Citation Extraction
 - Evidence Validation
 - Hallucination Auditing
-

14.16 Document Citation Relationship

Relationship

Document

↓

Citation

Purpose

Support:

- Traceability
 - Provenance
 - Verification
-

14.17 Document Retrieval Relationship

Relationship

Document
↓
Retrieval Result

Purpose

Support:

BM25 Retrieval
Vector Retrieval
Hybrid Retrieval
Reranking

14.18 Document Audit Model

Every document must support:

Document Audit
Chunk Audit
Evidence Audit
Citation Audit
Retrieval Audit

Audit records remain immutable.

14.19 Document Integrity Rules

Foreign key constraints must enforce:

Document Integrity

Chunk Integrity

Metadata Integrity

Evidence Integrity

Citation Integrity

Orphan records are not allowed.

14.20 Section 14 Acceptance Criteria

Section 14 is complete only if:

- ✓ Document storage is supported
 - ✓ Document versioning is supported
 - ✓ Chunk storage is supported
 - ✓ Chunk relationships are supported
 - ✓ Metadata extraction is supported
 - ✓ Recursive chunking is supported
 - ✓ Retrieval linkage is supported
 - ✓ Evidence linkage is supported
 - ✓ Auditability is supported
 - ✓ Enterprise RAG ingestion is supported
-

DDD V1 – Section 15

Vector Search Domain Architecture

15.1 Purpose

The Vector Search Domain is responsible for storing and managing:

- Embeddings
- Embedding Versions
- Vector Indexes
- Similarity Searches
- Semantic Retrieval Results
- Vector Search Audit Records

The Vector Search Domain enables semantic retrieval within TODP's Enterprise RAG architecture.

Its purpose is to support:

- Semantic Search
 - Similarity Search
 - Context Search
 - Hybrid Retrieval
 - Recommendation Discovery
-

15.2 Vector Search Philosophy

TODP follows:

```
Documents
↓
Chunks
↓
Embeddings
↓
Vector Retrieval
↓
Evidence
↓
Recommendation
```

and

Vector Search
≠
Recommendation

Vector search retrieves candidates.

Evidence validation and ranking happen later.

15.3 Vector Search Entity Diagram

```
embeddings
├── embedding_versions
├── vector_indexes
├── similarity_searches
├── retrieval_results
├── chunks
├── documents
└── evidence
```

15.4 Embeddings Table

Primary Entity

embeddings

Purpose

Store vector embeddings.

Primary Key

embedding_id

Core Fields

embedding_id

chunk_id

embedding_model
embedding_dimension
embedding_vector
created_at
version

Rule

Every embedding must belong to exactly one chunk.

15.5 Embedding Versions Table

Primary Entity

embedding_versions

Purpose

Store embedding history.

Core Fields

embedding_version_id
embedding_id
model_version
effective_date
change_reason
created_at

Rule

Historical embeddings must never be overwritten.

15.6 Vector Indexes Table

Primary Entity

`vector_indexes`

Purpose

Store vector index metadata.

Core Fields

`index_id`

`index_name`

`index_type`

`embedding_model`

`created_at`

`version`

Examples

`pgvector_ivfflat`

`pgvector_hnsw`

`future_distributed_index`

15.7 Similarity Searches Table

Primary Entity

`similarity_searches`

Purpose

Store semantic search executions.

Core Fields

search_id
query_embedding_id
index_id
search_timestamp
search_status
created_at
version

15.8 Similarity Results Table

Primary Entity

similarity_results

Purpose

Store vector search outputs.

Core Fields

result_id
search_id
chunk_id
similarity_score
rank_position
created_at
version

Rule

Similarity results are candidates.

They are not evidence.

They are not recommendations.

15.9 Embedding Generation Workflow

```
Document
↓
Chunk
↓
Embedding Generation
↓
Embedding Validation
↓
Storage
↓
Indexing
```

15.10 Semantic Retrieval Workflow

```
Query
↓
Query Embedding
↓
Vector Search
↓
Similarity Results
↓
Evidence Extraction
```

15.11 pgvector Architecture

V1 Technology Stack

```
PostgreSQL
+
pgvector
```

Purpose

Support:

Embedding Storage

Similarity Search

15.12 Hybrid Retrieval Integration

TODP supports:

BM25
+
Vector Search
+
Metadata Filters

Workflow

BM25 Results
+
Vector Results
↓
Hybrid Merge
↓
Reranking
↓
Evidence Pipeline

15.13 Chunk Relationship Model

Relationship

Chunk
↓
Embedding

Rule

One chunk may have:

Many Embedding Versions

One active embedding at a time.

15.14 Evidence Relationship Model

Relationship

Embedding
↓
Retrieved Chunk
↓
Evidence

Purpose

Support:

- Recommendation Traceability
 - Hallucination Detection
 - Retrieval Reproducibility
-

15.15 Similarity Scoring Model

Stored Fields

Similarity Score
Embedding Version
Retrieval Timestamp

Purpose

Allow retrieval reproducibility.

15.16 Vector Audit Model

Every vector operation must support:

Embedding Audit
Search Audit

Retrieval Audit

Evidence Audit

Audit records remain immutable.

15.17 Vector Reproducibility Model

The system must support:

Same Query

Same Embedding Model

Same Index

Same Version

↓

Same Retrieval Result

Purpose

Ensure reproducible retrieval behavior.

15.18 Vector Integrity Rules

Foreign key constraints must enforce:

Embedding Integrity

Chunk Integrity

Document Integrity

Search Integrity

Result Integrity

Orphan records are not allowed.

15.19 Vector Scalability Rules

The Vector Search Domain must support:

Millions of Embeddings

Millions of Chunks

Millions of Searches

Millions of Similarity Results

without redesigning the architecture.

Future migration path:

PostgreSQL + pgvector

↓

Qdrant

PostgreSQL + pgvector

↓

Weaviate

PostgreSQL + pgvector

↓

Distributed Vector Infrastructure

Business logic must remain unchanged.

15.20 Section 15 Acceptance Criteria

Section 15 is complete only if:

- ✓ Embedding storage is supported
- ✓ Embedding versioning is supported
- ✓ Vector indexes are supported

- ✓ Similarity search is supported
 - ✓ Similarity scoring is supported
 - ✓ Hybrid retrieval integration is supported
 - ✓ Retrieval reproducibility is supported
 - ✓ Auditability is supported
 - ✓ Traceability is supported
 - ✓ Enterprise RAG vector retrieval is supported
-

DDD V1 – Section 16

Registration Domain Architecture

16.1 Purpose

The Registration Domain is responsible for storing and managing:

- Registrations
- Registration Workflows
- Registration Requirements
- Registration Status
- Registration Audit Records
- Registration Versions

The Registration Domain assists users in completing registrations for:

- Scholarships
- Events
- Workshops
- Courses
- Government Programs
- Community Initiatives

TODP assists registration.

TODP does not submit registrations automatically.

16.2 Registration Domain Philosophy

TODP follows:

Assist
Do Not Auto Submit

and

Human Approval
Required

The platform may:

Auto Fill

Suggest Documents

Suggest Requirements

Validate Completeness

The platform must not:

Auto Submit

Auto Approve

Auto Sign

Auto Accept Terms

16.3 Registration Domain Entity Diagram

```
registrations
├── registration_requirements
├── registration_documents
├── registration_status
├── registration_versions
├── users
├── opportunities
└── audit_logs
```

16.4 Registrations Table

Primary Entity

registrations

Purpose

Store registration records.

Primary Key

registration_id

Core Fields

registration_id

user_id

opportunity_id

registration_status

created_at

updated_at

version

Supported Types

Scholarship Registration

Program Registration

Course Registration

Event Registration

Workshop Registration

16.5 Registration Requirements Table

Primary Entity

registration_requirements

Purpose

Store registration prerequisites.

Core Fields

requirement_id

opportunity_id

requirement_text

source_id

citation_id

created_at

version

Examples

Eligibility

Age Requirement

Identity Proof

Academic Proof

Fee Requirement

Rule

Requirements must reference evidence.

16.6 Registration Documents Table

Primary Entity

registration_documents

Purpose

Store registration-related document references.

Core Fields

document_id

registration_id

document_type

document_status

created_at

version

Examples

Resume

Certificate

Transcript

Identity Proof

Application Form

16.7 Registration Status Table

Primary Entity

registration_status

Purpose

Store registration progress.

Core Fields

status_id
registration_id
status_name
status_timestamp
created_at

Supported Status

Draft
Ready
Submitted
Accepted
Rejected
Withdrawn

16.8 Registration Versions Table

Primary Entity

registration_versions

Purpose

Store historical registration records.

Core Fields

registration_version_id
registration_id
version_number

effective_date

change_reason

created_at

Rule

Historical records must never be overwritten.

16.9 Registration Workflow

Opportunity

↓

Requirements Review

↓

Document Collection

↓

Auto Fill Assistance

↓

User Review

↓

User Approval

↓

Submission

16.10 Auto Fill Model

TODP may assist with:

Name

Email

Phone

Address

Academic Details

Only when:

User Consent Exists

16.11 Auto Submit Restriction

TODP V1 Rule

Auto Submit

NOT ALLOWED

Every submission requires:

Explicit User Action

16.12 Registration Evidence Relationship

Relationship

Registration

↓

Requirements

↓

Evidence

↓

Citation

↓

Source

Purpose

Support registration traceability.

16.13 Registration Audit Model

Every registration must support:

Requirement Audit

Evidence Audit

Submission Audit

Version Audit

Audit records remain immutable.

16.14 Registration Recommendation Model

Relationship

Recommendation
↓
Registration

Purpose

Allow users to act on recommendations.

Example

Scholarship Recommended
↓
Registration Assistance

16.15 Registration Discovery Hierarchy

Registration opportunities follow:

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

Purpose

Minimize:

Money Cost

Time Cost

Distance Cost

before expanding outward.

16.16 Registration Traceability Model

Every registration must answer:

What was registered?

Why was it recommended?

What requirements existed?

What evidence supported it?

What source defined it?

16.17 Registration Governance Rules

The system must continuously answer:

Can every registration be traced?

Can every requirement be verified?

Can every submission be audited?

Can every registration be reproduced?

These rules are mandatory.

16.18 Registration Integrity Rules

Foreign key constraints must enforce:

Registration Integrity

User Integrity

Opportunity Integrity

Evidence Integrity

Audit Integrity

Orphan records are not allowed.

16.19 Registration Scalability Rules

The Registration Domain must support:

Millions of Registrations

Millions of Users

Millions of Opportunities

Historical Versions

without redesigning the architecture.

16.20 Section 16 Acceptance Criteria

Section 16 is complete only if:

- ✓ Registration storage is supported
- ✓ Registration requirements are supported
- ✓ Registration documents are supported
- ✓ Registration status tracking is supported
- ✓ Auto fill assistance is supported
- ✓ Auto submit prevention is supported
- ✓ Auditability is supported

- ✓ Traceability is supported
 - ✓ Reproducibility is supported
 - ✓ Recommendation linkage is supported
-

DDD V1 – Section 17

Calendar Domain Architecture

17.1 Purpose

The Calendar Domain is responsible for storing and managing:

- Calendar Entries
- Calendar Availability
- Scheduling Assistance
- Event Scheduling
- Reminder Scheduling
- Calendar Audit Records
- Calendar Versions

The Calendar Domain exists to help users organize discovered opportunities.

TODP assists scheduling.

TODP does not make scheduling decisions for users.

17.2 Calendar Domain Philosophy

TODP follows:

Assist Scheduling

Do Not Control Scheduling

and

User Consent Required

The system may:

Read Availability

Suggest Time Slots

Create Draft Entries

Generate Reminders

The system must not:

Modify Calendar Without Consent

Delete Calendar Events

Accept Invitations

Commit User Availability

17.3 Calendar Domain Entity Diagram

```
calendar_entries
├── calendar_availability
├── calendar_reminders
├── calendar_versions
├── users
├── opportunities
└── audit_logs
```

17.4 Calendar Entries Table

Primary Entity

calendar_entries

Purpose

Store scheduled opportunity records.

Primary Key

calendar_entry_id

Core Fields

calendar_entry_id

user_id

opportunity_id

event_title

start_time

end_time

calendar_status

created_at

updated_at

version

Examples

Workshop Schedule

Course Session

Scholarship Deadline

Program Interview

Community Event

17.5 Calendar Availability Table

Primary Entity

calendar_availability

Purpose

Store user availability references.

Core Fields

availability_id
user_id
availability_start
availability_end
availability_type
created_at
version

Examples

Available
Busy
Tentative
Blocked

17.6 Calendar Reminders Table

Primary Entity

calendar_reminders

Purpose

Store reminder configurations.

Core Fields

reminder_id
calendar_entry_id
reminder_time
reminder_type
created_at
version

Examples

Email Reminder

Push Notification

SMS Reminder

In-App Reminder

17.7 Calendar Versions Table

Primary Entity

`calendar_versions`

Purpose

Store historical calendar records.

Core Fields

`calendar_version_id`

`calendar_entry_id`

`version_number`

`effective_date`

`change_reason`

`created_at`

Rule

Historical calendar records must never be overwritten.

17.8 Calendar Integration Model

Supported Integrations

Google Calendar

Microsoft Outlook

Apple Calendar

Internal TODP Calendar

Integration is optional.

User consent is mandatory.

17.9 Calendar Scheduling Workflow

Opportunity

↓

User Interest

↓

Availability Check

↓

Schedule Suggestion

↓

User Approval

↓

Calendar Entry

17.10 Opportunity Calendar Relationship

Relationship

Opportunity

↓

Calendar Entry

Examples

Event

↓

Schedule

Workshop

↓

Schedule

Course

↓

Schedule

17.11 Registration Calendar Relationship

Relationship

Registration

↓

Calendar Entry

Purpose

Track:

Deadlines

Interviews

Submission Dates

Review Dates

17.12 Calendar Recommendation Model

Relationship

Recommendation

↓

Calendar Entry

Purpose

Allow users to schedule recommended opportunities.

Example

Scholarship Recommended

↓

Deadline Scheduled

17.13 Calendar Availability Model

The system may answer:

Is the user available?

Does this conflict?

What time slot is free?

The system must not answer:

What should the user prioritize?

Priority remains user-controlled.

17.14 Calendar Discovery Hierarchy

Calendar scheduling supports:

Institution

Local

District

State

National

International

Purpose

Schedule opportunities regardless of discovery scope.

17.15 Calendar Audit Model

Every calendar action must support:

Schedule Audit

Reminder Audit

Availability Audit

Version Audit

Audit records remain immutable.

17.16 Calendar Traceability Model

Every calendar entry must answer:

What opportunity created this entry?

When was it scheduled?

Who approved it?

What recommendation triggered it?

What registration is associated with it?

17.17 Calendar Governance Rules

The system must continuously answer:

Can every schedule be traced?

Can every reminder be audited?

Can every availability check be reproduced?

Can every calendar entry be explained?

These rules are mandatory.

17.18 Calendar Integrity Rules

Foreign key constraints must enforce:

Calendar Integrity

User Integrity

Opportunity Integrity

Registration Integrity

Audit Integrity

Orphan records are not allowed.

17.19 Calendar Scalability Rules

The Calendar Domain must support:

Millions of Calendar Entries

Millions of Reminders

Millions of Availability Records

Historical Versions

without redesigning the architecture.

17.20 Section 17 Acceptance Criteria

Section 17 is complete only if:

- ✓ Calendar entries are supported
- ✓ Availability tracking is supported
- ✓ Reminder management is supported
- ✓ Calendar integrations are supported

- ✓ User approval workflow is supported
 - ✓ Auditability is supported
 - ✓ Traceability is supported
 - ✓ Reproducibility is supported
 - ✓ Registration linkage is supported
 - ✓ Recommendation linkage is supported
-

DDD V1 – Section 18

Agent Architecture Domain

18.1 Purpose

The Agent Architecture Domain defines all system agents responsible for:

- Discovery
- Retrieval
- Validation
- Ranking
- Recommendation
- Registration Assistance
- Calendar Assistance
- Auditing
- Hallucination Detection

The objective is to create an evidence-first multi-agent architecture where every decision is traceable, auditable and reproducible.

18.2 Agent Architecture Philosophy

TODP follows:

Retrieve First

Validate Second

Rank Third

Recommend Fourth

Audit Continuously

and

No Evidence

=

No Strong Recommendation

Agents do not create facts.

Agents discover, validate, rank and explain evidence.

18.3 Agent Architecture Overview

User

↓

Discovery Agent

↓

Authority Agent

↓

Retrieval Agent

↓

Evidence Agent

↓

Ranking Agent

↓

Recommendation Agent

↓

Audit Agents

↓

User

18.4 Agent Registry

TODP V1 contains:

Discovery Agent

Authority Discovery Agent

Retrieval Agent

Evidence Agent

Ranking Agent

Recommendation Agent

Citation Agent

Registration Agent

Calendar Agent

Audit Agent

Hallucination Audit Agent

Every agent is independently auditable.

18.5 Discovery Agent

Primary Responsibility

Discover Opportunities

Supports:

Scholarships

Programs

Courses

Events

Workshops

Workflow

User Query
↓
Discovery Scope
↓
Candidate Retrieval
↓
Authority Validation

18.6 Authority Discovery Agent

Primary Responsibility

Discover Authorities

Supports:

Government Bodies

Universities

Departments

Regulators

Organizations

Workflow

Source Discovery
↓
Authority Extraction
↓
Hierarchy Discovery
↓
Validation

18.7 Retrieval Agent

Primary Responsibility

Retrieve Evidence Candidates

Supports:

BM25 Retrieval
Vector Retrieval
Hybrid Retrieval
Metadata Filtering

Workflow

Query
↓
Retrieval
↓
Candidate Results

18.8 Evidence Agent

Primary Responsibility

Create Evidence Records

Supports:

Evidence Validation
Evidence Creation
Evidence Linking
Evidence Chains

Workflow

Retrieved Result
↓
Evidence Creation
↓
Citation Mapping
↓
Source Mapping

18.9 Ranking Agent

Primary Responsibility

Generate Rankings

V1 Ranking Factors

Money Cost

Time Cost

Distance Cost

Workflow

Evidence

↓

Cost Calculation

↓

Ranking

Rule

Lowest total cost ranks highest.

18.10 Recommendation Agent

Primary Responsibility

Generate Recommendations

Workflow

Ranking

↓

Recommendation

↓

Explanation

Rule

Every recommendation must reference:

Evidence

Citation

Source

Ranking Run

18.11 Citation Agent

Primary Responsibility

Validate Citations

Responsibilities

Citation Discovery

Citation Validation

Citation Mapping

Citation Audit

Rule

Invalid citations cannot be used.

18.12 Registration Agent

Primary Responsibility

Registration Assistance

Supports

Auto Fill

Requirement Discovery

Document Collection

Restriction

Auto Submit

NOT ALLOWED

18.13 Calendar Agent

Primary Responsibility

Scheduling Assistance

Supports

Availability Checks

Reminder Creation

Calendar Draft Entries

User approval required.

18.14 Audit Agent

Primary Responsibility

Continuous Auditing

Responsibilities

Recommendation Audit

Ranking Audit

Citation Audit

Evidence Audit

Source Audit

18.15 Hallucination Audit Agent

Primary Responsibility

Detect Unsupported Outputs

Responsibilities

Claim Validation

Evidence Validation

Source Validation

Citation Validation

Rule

Unsupported outputs must be flagged.

18.16 Agent Communication Architecture

Agent

↓

Message

↓

Task

↓

Result

↓

Audit Record

Every agent interaction must be logged.

18.17 Agent Audit Architecture

Every agent execution must store:

Agent ID

Task ID

Input

Output

Timestamp

Version

Audit Record

Purpose

Support reproducibility.

18.18 Agent Governance Rules

The system must continuously answer:

Can every agent decision be traced?

Can every agent output be audited?

Can every agent action be reproduced?

Can every agent result be explained?

These rules are mandatory.

18.19 Agent Scalability Rules

The Agent Architecture must support:

Single Agent Execution

Parallel Agents

Distributed Agents

Future Agent Expansion

without redesigning the platform.

18.20 Section 18 Acceptance Criteria

Section 18 is complete only if:

- ✓ Discovery Agent is supported
 - ✓ Authority Discovery Agent is supported
 - ✓ Retrieval Agent is supported
 - ✓ Evidence Agent is supported
 - ✓ Ranking Agent is supported
 - ✓ Recommendation Agent is supported
 - ✓ Citation Agent is supported
 - ✓ Registration Agent is supported
 - ✓ Calendar Agent is supported
 - ✓ Audit & Hallucination Audit Agents are supported
-

DDD V1 – Section 19

Observability & Monitoring Domain Architecture

19.1 Purpose

The Observability & Monitoring Domain is responsible for continuously monitoring the behavior, performance, health, traceability, auditability and reliability of the TODP platform.

The purpose of this domain is to answer:

Is the system working?

Is the system healthy?

Is the system traceable?

Is the system auditable?

Is the system hallucinating?

Is the system reproducible?

This domain is the operational backbone of TODP.

19.2 Observability Philosophy

TODP follows:

Trace Everything

Audit Continuously

Monitor Continuously

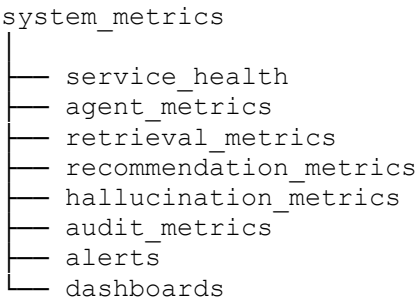
and

What Cannot Be Observed

Cannot Be Trusted

Every critical operation must be observable.

19.3 Observability Domain Entity Diagram



19.4 System Metrics Table

Primary Entity

`system_metrics`

Purpose

Store platform-wide metrics.

Primary Key

`metric_id`

Core Fields

- `metric_id`
- `metric_name`
- `metric_value`
- `metric_timestamp`
- `metric_source`
- `created_at`

Examples

CPU Usage

Memory Usage

Database Latency

API Latency

Query Throughput

19.5 Service Health Table

Primary Entity

service_health

Purpose

Monitor platform services.

Core Fields

service_health_id

service_name

health_status

response_time

checked_at

created_at

Supported Status

Healthy

Warning

Degraded

Failed

19.6 Agent Metrics Table

Primary Entity

agent_metrics

Purpose

Monitor agent performance.

Core Fields

agent_metric_id

agent_name

execution_count

average_runtime

error_count

created_at

Monitored Agents

Discovery Agent

Authority Agent

Retrieval Agent

Evidence Agent

Ranking Agent

Recommendation Agent

Citation Agent

Registration Agent

Calendar Agent

Audit Agent

Hallucination Audit Agent

19.7 Retrieval Metrics Table

Primary Entity

retrieval_metrics

Purpose

Monitor retrieval quality.

Core Fields

retrieval_metric_id

retrieval_run_id

retrieval_latency

retrieval_count

retrieval_timestamp

Examples

BM25 Latency

Vector Latency

Hybrid Latency

Rerank Latency

19.8 Recommendation Metrics Table

Primary Entity

recommendation_metrics

Purpose

Monitor recommendation quality.

Core Fields

recommendation_metric_id
recommendation_id
generation_time
evidence_count
citation_count
created_at

Examples

Recommendations Generated
Recommendations Audited
Recommendations Failed Audit

19.9 Hallucination Metrics Table

Primary Entity

hallucination_metrics

Purpose

Monitor hallucination behavior.

Core Fields

hallucination_metric_id
hallucination_check_id
claim_count
failure_count
created_at

Examples

Unsupported Claims
Missing Citations
Missing Sources
Unsupported Recommendations

19.10 Audit Metrics Table

Primary Entity

`audit_metrics`

Purpose

Monitor audit quality.

Core Fields

`audit_metric_id`
`audit_id`
`audit_status`
`audit_duration`
`created_at`

Examples

Audit Success Rate
Audit Failure Rate
Average Audit Time

19.11 Alerting System

Primary Entity

`alerts`

Purpose

Store operational alerts.

Core Fields

alert_id

alert_type

severity

alert_message

alert_timestamp

alert_status

Supported Severity

Low

Medium

High

Critical

19.12 Dashboard Domain

Primary Entity

dashboards

Purpose

Support operational visibility.

Examples

System Dashboard

Agent Dashboard

Retrieval Dashboard

Recommendation Dashboard

Hallucination Dashboard

Audit Dashboard

19.13 Recommendation Monitoring Model

The platform continuously measures:

Recommendation Volume

Recommendation Audit Pass Rate

Recommendation Failure Rate

Recommendation Reproducibility

Purpose

Ensure recommendation quality.

19.14 Hallucination Monitoring Model

The platform continuously measures:

Hallucination Rate

Unsupported Claims

Missing Citations

Missing Sources

Purpose

Reduce hallucination risk.

19.15 Retrieval Monitoring Model

The platform continuously measures:

Retrieval Latency

Retrieval Accuracy

Retrieval Throughput

Retrieval Reproducibility

Purpose

Ensure retrieval quality.

19.16 Agent Monitoring Model

Every agent execution must store:

Agent ID

Task ID

Execution Time

Status

Audit Record

Purpose

Support operational debugging.

19.17 Observability Governance Rules

The platform must continuously answer:

Can every recommendation be monitored?

Can every ranking be monitored?

Can every retrieval be monitored?

Can every hallucination be monitored?

Can every audit be monitored?

These rules are mandatory.

19.18 Observability Integrity Rules

Foreign key constraints must enforce:

Metric Integrity

Alert Integrity

Audit Integrity

Recommendation Integrity

Retrieval Integrity

Orphan records are not allowed.

19.19 Observability Scalability Rules

The Observability Domain must support:

Billions of Metrics

Millions of Alerts

Millions of Audit Events

Long-Term Historical Storage

without redesigning the architecture.

Recommended V1 Stack

OpenTelemetry

Prometheus

Grafana

PostgreSQL

All are open-source and compatible with a zero-cost MVP.

19.20 Section 19 Acceptance Criteria

Section 19 is complete only if:

- ✓ System monitoring is supported
 - ✓ Service health monitoring is supported
 - ✓ Agent monitoring is supported
 - ✓ Retrieval monitoring is supported
 - ✓ Recommendation monitoring is supported
 - ✓ Hallucination monitoring is supported
 - ✓ Audit monitoring is supported
 - ✓ Alerting is supported
 - ✓ Dashboarding is supported
 - ✓ Enterprise observability is supported
-

DDD V1 – Section 20

API & Integration Domain Architecture

20.1 Purpose

The API & Integration Domain is responsible for exposing TODP capabilities to:

- Web Applications
- Mobile Applications
- Agents
- External Systems
- Calendar Providers
- Registration Providers
- Future Partner Integrations

The API layer acts as the controlled gateway into the TODP platform.

Every API operation must be:

- Traceable
 - Auditable
 - Versioned
 - Reproducible
-

20.2 API Philosophy

TODP follows:

API First

Evidence First

Audit First

and

Every API Request
Must Be Traceable

The API layer is not allowed to bypass:

- Evidence Validation
 - Audit Logging
 - Traceability Logging
-

20.3 API Domain Entity Diagram

```
api_clients
├── api_keys
├── api_requests
├── api_responses
├── api_versions
├── integrations
├── webhooks
└── audit_logs
```

20.4 API Clients Table

Primary Entity

api_clients

Purpose

Store API consumer records.

Primary Key

client_id

Core Fields

client_id
client_name
client_type
client_status
created_at
version

Examples

Web Frontend

Mobile App

Internal Agent

Partner System

20.5 API Keys Table

Primary Entity

api_keys

Purpose

Store authentication credentials.

Core Fields

api_key_id

client_id

key_hash

key_status

created_at

expires_at

Supported Status

Active

Revoked

Expired

20.6 API Requests Table

Primary Entity

`api_requests`

Purpose

Store incoming API requests.

Core Fields

`request_id`

`client_id`

`endpoint`

`request_timestamp`

`request_status`

`trace_id`

Purpose

Support:

- Traceability
 - Monitoring
 - Auditing
-

20.7 API Responses Table

Primary Entity

`api_responses`

Purpose

Store API responses.

Core Fields

`response_id`
`request_id`
`response_status`
`response_timestamp`
`trace_id`

Rule

Every response must be linked to a request.

20.8 API Versions Table

Primary Entity

`api_versions`

Purpose

Store API version history.

Core Fields

`api_version_id`
`version_name`
`effective_date`
`change_reason`
`created_at`

Rule

APIs must be versioned.

20.9 Integration Registry Table

Primary Entity

integrations

Purpose

Store external integration definitions.

Core Fields

integration_id

integration_name

integration_type

integration_status

created_at

version

Examples

Google Calendar

Microsoft Outlook

University Portal

Government Portal

Future Partner APIs

20.10 Webhooks Table

Primary Entity

webhooks

Purpose

Store outbound event subscriptions.

Core Fields

webhook_id

client_id

event_type

target_url

status

created_at

Examples

Registration Updated

Calendar Updated

Recommendation Generated

Audit Completed

20.11 Discovery APIs

Core Endpoints

GET /opportunities

GET /events

GET /courses

GET /programs

GET /scholarships

Purpose

Support opportunity discovery.

20.12 Recommendation APIs

Core Endpoints

GET /recommendations

GET /recommendations/{id}

GET /recommendations/{id}/evidence

GET /recommendations/{id}/audit

Purpose

Support recommendation traceability.

20.13 Retrieval APIs

Core Endpoints

POST /search

POST /retrieve

GET /retrieval-runs

GET /retrieval-runs/{id}

Purpose

Support Enterprise RAG retrieval.

20.14 Registration APIs

Core Endpoints

POST /registrations

GET /registrations

GET /registrations/{id}

POST /registrations/autofill

Restriction

No Auto Submit Endpoint

20.15 Calendar APIs

Core Endpoints

GET /calendar

POST /calendar/events

POST /calendar/reminders

GET /calendar/availability

User consent required.

20.16 Audit APIs

Core Endpoints

GET /audits

GET /audits/{id}

GET /hallucination-audits

GET /ranking-audits

Purpose

Support transparency.

20.17 API Traceability Model

Every API request must store:

Trace ID

Request ID

Timestamp

Client ID

Version

Purpose

Support reproducibility.

20.18 API Governance Rules

The platform must continuously answer:

Can every API request be traced?

Can every API response be audited?

Can every API result be reproduced?

Can every API action be monitored?

These rules are mandatory.

20.19 API Scalability Rules

The API Domain must support:

Millions of Requests

Millions of Responses

Thousands of Clients

Future Partner Integrations

without redesigning the architecture.

Recommended V1 Stack

FastAPI

Pydantic

PostgreSQL

Redis

Open-source and compatible with a zero-cost MVP.

20.20 Section 20 Acceptance Criteria

Section 20 is complete only if:

- ✓ API clients are supported
- ✓ API authentication is supported
- ✓ API request logging is supported
- ✓ API response logging is supported
- ✓ API versioning is supported
- ✓ Integration registry is supported
- ✓ Webhooks are supported
- ✓ Audit APIs are supported
- ✓ Traceability is supported
- ✓ Enterprise integration is supported

DDD V1 – Section 21

Security & Access Control Domain Architecture

Status: READY FOR REVIEW

21.1 Purpose

The Security & Access Control Domain is responsible for protecting:

- Users
- Data
- APIs
- Recommendations
- Evidence
- Audit Records
- Agent Operations

The purpose of this domain is to ensure:

- Confidentiality
- Integrity
- Availability
- Traceability
- Auditability

without violating TODP's Evidence-First architecture.

21.2 Security Philosophy

TODP follows:

Trust Evidence

Not Access

and

Every Access

Must Be Traceable

and

Every Permission

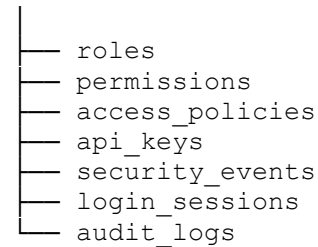
Must Be Auditable

Security decisions must be logged.

Security actions must be reproducible.

21.3 Security Domain Entity Diagram

users



21.4 Security Architecture Model

TODP V1 uses:

Authentication

+

Authorization

+

Audit Logging

+

Encryption

Security is enforced at:

API Layer

Agent Layer

Database Layer

21.5 Roles Table

Primary Entity

roles

Purpose

Store role definitions.

Core Fields

role_id

role_name

role_description

created_at

version

Examples

User

Administrator

Auditor

System Agent

Platform Operator

21.6 Permissions Table

Primary Entity

permissions

Purpose

Store permission definitions.

Core Fields

permission_id
permission_name
permission_scope
created_at
version

Examples

Read Evidence
Read Recommendation
Read Audit
Create Registration
Manage Users

21.7 Role Permission Mapping

Primary Entity

role_permissions

Purpose

Assign permissions to roles.

Relationship

Role
↓
Permissions

Rule

Permissions must be explicit.

Implicit permissions are not allowed.

21.8 Access Policies Table

Primary Entity

`access_policies`

Purpose

Store access control rules.

Core Fields

`policy_id`

`policy_name`

`policy_type`

`policy_status`

`created_at`

`version`

Examples

Evidence Access Policy

Audit Access Policy

API Access Policy

Admin Access Policy

21.9 Login Sessions Table

Primary Entity

`login_sessions`

Purpose

Track authenticated sessions.

Core Fields

`session_id`

`user_id`

`login_time`

`logout_time`

`session_status`

`created_at`

Supported Status

Active

Expired

Revoked

21.10 Security Events Table

Primary Entity

`security_events`

Purpose

Store security activities.

Core Fields

event_id
event_type
event_description
severity
event_timestamp

Examples

Login
Logout
Permission Change
Policy Change
Failed Authentication
Access Denied

21.11 Authentication Architecture

V1 Recommended Authentication

Email Authentication
OAuth Authentication
JWT Tokens

Supported Providers

Google
Microsoft
Email Login

21.12 Authorization Architecture

V1 Authorization Model

RBAC

Role-Based Access Control.

Relationship

User

↓

Role

↓

Permission

Future Versions

RBAC

+

ABAC

Attribute-Based Access Control.

21.13 Data Protection Model

Sensitive Data

Email

Location

Calendar Data

Registration Data

Session Data

Protection Requirements

Encryption At Rest

21.14 API Security Model

Every API request must contain:

Authentication

Authorization

Trace ID

Audit Record

Unauthorized access must be rejected.

21.15 Agent Security Model

Every agent execution must store:

Agent ID

Task ID

Permission Scope

Execution Timestamp

Purpose

Support:

- Agent Auditing
 - Agent Governance
-

21.16 Security Audit Model

Every security event must support:

Authentication Audit

Authorization Audit

Policy Audit

Permission Audit

Audit records remain immutable.

21.17 Security Governance Rules

The platform must continuously answer:

Who accessed the data?

When was it accessed?

Why was it accessed?

What permissions were used?

Can access be reproduced?

These rules are mandatory.

21.18 Security Integrity Rules

Foreign key constraints must enforce:

Role Integrity

Permission Integrity

Policy Integrity

Session Integrity

Audit Integrity

Orphan records are not allowed.

21.19 Security Scalability Rules

The Security Domain must support:

Millions of Users

Millions of Sessions

Millions of Security Events

Historical Audit Storage

without redesigning the architecture.

Recommended V1 Stack

JWT

OAuth2

FastAPI Security

PostgreSQL

HTTPS

Open-source and suitable for zero-cost MVP deployment.

21.20 Section 21 Acceptance Criteria

Section 21 is complete only if:

- ✓ Authentication is supported
- ✓ Authorization is supported
- ✓ RBAC is supported
- ✓ Session management is supported

- ✓ Security event logging is supported
 - ✓ API security is supported
 - ✓ Agent security is supported
 - ✓ Security auditing is supported
 - ✓ Encryption requirements are defined
 - ✓ Enterprise-grade access control is supported
-

DDD V1 – Section 22

Data Governance Domain Architecture

22.1 Purpose

The Data Governance Domain is responsible for managing:

- Data Ownership
- Data Quality
- Data Lineage
- Data Retention
- Data Classification
- Data Lifecycle
- Data Compliance

The purpose of this domain is to ensure that every piece of data inside TODP remains:

- Traceable
- Auditable
- Reproducible
- Governed

throughout its lifecycle.

22.2 Data Governance Philosophy

TODP follows:

Trace Everything

Govern Everything

and

Every Data Element

Must Have Ownership

and

Every Data Element

Must Have Lineage

No data should exist without:

- Source
- Owner
- Lineage
- Audit Trail

22.3 Data Governance Entity Diagram

```
data_assets
├── data_classifications
├── data_lineage
├── data_retention_policies
├── data_quality_checks
├── data_stewards
├── audit_logs
└── compliance_records
```

22.4 Data Assets Table

Primary Entity

data_assets

Purpose

Store governance metadata for all platform data.

Primary Key

data_asset_id

Core Fields

data_asset_id

asset_name

asset_type

owner_id

classification_id

created_at

version

Examples

User Data

Opportunity Data

Evidence Data

Citation Data

Audit Data

Recommendation Data

22.5 Data Classification Table

Primary Entity

data_classifications

Purpose

Classify platform data.

Core Fields

classification_id

classification_name

classification_level

created_at

version

Supported Levels

Public

Internal

Restricted

Confidential

22.6 Data Steward Table

Primary Entity

data_stewards

Purpose

Define data ownership.

Core Fields

steward_id

asset_id

owner_role

owner_name

created_at

Examples

System Owner

Platform Administrator

Data Auditor

22.7 Data Lineage Table

Primary Entity

data_lineage

Purpose

Track data origins and transformations.

Core Fields

lineage_id

source_asset_id

target_asset_id

transformation_type

created_at

Example

Source

↓

Document

↓

Chunk

↓

Evidence

↓

22.8 Data Retention Policies Table

Primary Entity

`data_retention_policies`

Purpose

Define data retention rules.

Core Fields

`policy_id`

`asset_type`

`retention_period`

`retention_action`

`created_at`

Examples

Audit Logs → 7 Years

Recommendations → Permanent

Evidence → Permanent

Sessions → 90 Days

22.9 Data Quality Checks Table

Primary Entity

`data_quality_checks`

Purpose

Monitor data quality.

Core Fields

quality_check_id

asset_id

quality_type

quality_status

created_at

Examples

Completeness

Consistency

Validity

Uniqueness

Timeliness

22.10 Data Ownership Model

Every data asset must have:

Owner

Steward

Classification

Retention Policy

Rule

Unowned data assets are not allowed.

22.11 Data Lineage Model

Every recommendation must be traceable through:

Recommendation

↓

Evidence

↓

Citation

↓

Source

↓

Authority

Purpose

Support:

- Reproducibility
 - Auditability
 - Governance
-

22.12 Data Lifecycle Model

Supported Lifecycle

Creation

↓

Validation

↓

Storage

↓

Usage

↓

Audit

↓

Archival

↓

Deletion (if applicable)

Some records are permanent.

22.13 Data Quality Model

The platform continuously validates:

Missing Sources

Missing Citations

Broken Relationships

Invalid Metadata

Duplicate Records

Purpose

Maintain data integrity.

22.14 Data Compliance Model

The platform must maintain records for:

Consent

Audit Trails

Retention Rules

Access Logs

Policy Changes

Purpose

Support compliance readiness.

22.15 Data Governance Audit Model

Every governance action must support:

Ownership Audit

Lineage Audit

Retention Audit

Quality Audit

Compliance Audit

Audit records remain immutable.

22.16 Data Discovery Hierarchy Governance

TODP V1 Discovery Hierarchy

Institution

↓

Local

↓

District

↓

State

↓

National

↓

International

Purpose

Ensure governance consistency across all discovery scopes.

22.17 Data Governance Rules

The platform must continuously answer:

Who owns this data?

Where did this data originate?

How was this data transformed?

Can this data be audited?

Can this data be reproduced?

These rules are mandatory.

22.18 Data Integrity Rules

Foreign key constraints must enforce:

Asset Integrity

Ownership Integrity

Lineage Integrity

Retention Integrity

Audit Integrity

Orphan records are not allowed.

22.19 Data Governance Scalability Rules

The Data Governance Domain must support:

Billions of Records

Millions of Data Assets

Millions of Lineage Relationships

Long-Term Audit Storage

without redesigning the architecture.

Recommended V1 Stack

PostgreSQL

OpenMetadata

OpenLineage

Apache Atlas (Future)

DataHub (Future)

V1 may start with PostgreSQL-only governance metadata.

22.20 Section 22 Acceptance Criteria

Section 22 is complete only if:

- ✓ Data ownership is supported
- ✓ Data classification is supported
- ✓ Data lineage is supported
- ✓ Data retention policies are supported
- ✓ Data quality monitoring is supported
- ✓ Governance audits are supported
- ✓ Compliance records are supported
- ✓ Traceability is supported
- ✓ Auditability is supported
- ✓ Enterprise-grade data governance is supported

DDD V1 – Section 23

Authority Discovery Engine Architecture

23.1 Purpose

The Authority Discovery Engine is responsible for discovering, validating, maintaining and ranking authorities that exist within TODP.

The Authority Discovery Engine serves as the foundation for:

- Opportunity Discovery
- Evidence Validation
- Citation Validation
- Recommendation Generation
- Registration Assistance

Without authorities there can be no trusted recommendations.

23.2 Authority Discovery Philosophy

TODP follows:

Authority Before Recommendation

and

Discover Authorities

Do Not Invent Authorities

and

AI Is Not An Authority

Authorities must originate from observable systems.

23.3 Authority Discovery Engine Overview

Source Discovery

↓

Authority Discovery

↓

Authority Validation

↓

Hierarchy Discovery

↓

Evidence Mapping

↓

Recommendation Pipeline

23.4 Purpose of Authority Discovery

The engine exists to discover:

Government Bodies

Universities

Departments

Organizations

Foundations

Companies

Program Providers

Event Organizers

These entities become discoverable authorities.

23.5 Authority Registry

Primary Entity

`authority_registry`

Purpose

Store validated authorities.

Core Fields

authority_id
authority_name
authority_type
authority_level
authority_status
created_at
version

23.6 Authority Levels

TODP V1 Discovery Hierarchy

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

Purpose

Support cost-minimized discovery.

23.7 Authority Hierarchy Engine

Primary Entity

authority_hierarchy

Purpose

Store structural relationships.

Core Fields

hierarchy_id

parent_authority_id

child_authority_id

relationship_type

created_at

version

Examples

University

↓

Department

State Government

↓

Department

National Government

↓

Agency

International Organization

↓

Regional Office

23.8 Authority Discovery Sources

Authorities may originate from:

Official Websites

Government Portals

University Portals

Organization Directories

Public Registries

Program Websites

Authorities must be evidence-backed.

23.9 Authority Discovery Agent

Primary Responsibility

Discover Authorities

Workflow

Source

↓

Authority Extraction

↓

Validation

↓

Hierarchy Mapping

↓

Registry Update

23.10 Authority Validation Engine

Purpose

Verify discovered authorities.

Validation Questions

Does the authority exist?

Is the authority reachable?

Is the authority active?

Is the authority verifiable?

Does evidence support it?

23.11 Authority Evidence Model

Relationship

Authority
↓
Evidence
↓
Citation
↓
Source

Rule

Every authority must reference evidence.

23.12 Authority Discovery Workflow

Source Discovery
↓
Authority Discovery
↓
Evidence Validation
↓
Citation Validation
↓
Hierarchy Assignment
↓
Registry Storage

23.13 Authority Ranking Preparation

The Authority Discovery Engine itself does not rank.

It provides inputs for ranking.

Supported Authority Attributes

Authority Level

Location

Jurisdiction

Coverage Area

Provider Type

These attributes feed later ranking calculations.

23.14 Discovery Hierarchy Rule

TODP V1 requires:

Institution First

↓

Local

↓

District

↓

State

↓

National

↓

International

Purpose

Minimize:

Money Cost

Time Cost

Distance Cost

before expanding outward.

23.15 Authority Change Management

Authorities change over time.

Supported Changes

Name Changes

Structure Changes

Jurisdiction Changes

Department Changes

Status Changes

All changes must be versioned.

23.16 Authority Audit Model

Every authority must support:

Authority Audit

Evidence Audit

Citation Audit

Hierarchy Audit

Audit records remain immutable.

23.17 Authority Traceability Model

Every authority must answer:

Where was it discovered?

What evidence supports it?

What citations support it?

What source supports it?

What hierarchy contains it?

23.18 Authority Governance Rules

The platform must continuously answer:

Can every authority be verified?

Can every authority be audited?

Can every authority be traced?

Can every authority be reproduced?

These rules are mandatory.

23.19 Authority Scalability Rules

The Authority Discovery Engine must support:

Millions of Authorities

Millions of Hierarchy Relationships

Millions of Evidence Links

Historical Versions

without redesigning the architecture.

Recommended V1 Stack

PostgreSQL

Authority Registry Tables

Scheduled Discovery Jobs

Evidence Validation Pipeline

No external paid infrastructure required.

23.20 Section 23 Acceptance Criteria

Section 23 is complete only if:

- ✓ Authority registry is supported
 - ✓ Authority hierarchy is supported
 - ✓ Authority discovery is supported
 - ✓ Authority validation is supported
 - ✓ Evidence linkage is supported
 - ✓ Citation linkage is supported
 - ✓ Discovery hierarchy is supported
 - ✓ Auditability is supported
 - ✓ Traceability is supported
 - ✓ Enterprise-scale authority management is supported
-

DDD V1 – Section 24

Recommendation Engine Architecture

24.1 Purpose

The Recommendation Engine is responsible for generating evidence-backed recommendations for users.

The Recommendation Engine consumes:

- Authorities
- Sources
- Documents
- Chunks
- Citations
- Evidence
- Rankings

and produces:

- Opportunity Recommendations
- Registration Recommendations
- Calendar Recommendations
- Create Similar Recommendations
- Create New Recommendations

The Recommendation Engine is the primary user-facing intelligence layer of TODP.

24.2 Recommendation Engine Philosophy

TODP follows:

Retrieve First

Validate Second

Rank Third

Recommend Fourth

and

No Evidence

=

No Strong Recommendation

and

Recommendation
Requires
Ranking

Recommendations cannot bypass ranking.

24.3 Recommendation Engine Overview

User Query
↓
Authority Discovery
↓
Retrieval
↓
Evidence Creation
↓
Ranking
↓
Recommendation
↓
Audit
↓
User

24.4 Recommendation Modes

TODP V1 supports:

Discover Online

Discover Physical

Create Something Similar

Create Something New

All recommendation modes follow the same governance rules.

24.5 Discovery Recommendation Engine

Purpose

Recommend:

Scholarships

Programs

Courses

Events

Workshops

Workflow

Discovery Query

↓

Retrieval

↓

Ranking

↓

Recommendation

24.6 Discovery Hierarchy Engine

TODP V1 Discovery Order

Institution

↓

Local

↓

District

↓

State

↓

National

↓

International

Purpose

Minimize:

Money Cost

Time Cost

Distance Cost

before expanding outward.

24.7 Ranking Integration

The Recommendation Engine consumes outputs from:

Ranking Engine

V1 Ranking Formula

Total Cost

=

Money Cost

+

Time Cost

+

Distance Cost

Lowest total cost ranks highest.

24.8 Online Recommendation Model

For online opportunities:

Distance Cost = 0

Travel Time = 0

Ranking considers:

Money Cost

Access Time

Examples

Online Courses

Online Workshops

Online Events

Online Programs

24.9 Physical Recommendation Model

Examples

Workshops

Programs

Community Events

Physical Courses

Ranking considers:

Distance Cost

Travel Time

Money Cost

24.10 Create Similar Recommendation Engine

Purpose

Help users create opportunities similar to existing opportunities.

Workflow

Existing Opportunity

↓
Evidence Discovery
↓
Tool Discovery
↓
Resource Discovery
↓
Recommendation

Every recommendation must contain:

Evidence
Citation
Source
Audit Metadata

24.11 Create Similar Tool Ranking

Recommended tools are ranked using:

Money Cost
Time Cost
Distance Cost

Supported Sources

Institution Resources
Local Resources
District Resources
State Resources
National Resources
International Resources

24.12 Create New Recommendation Engine

Purpose

Help users create new opportunities.

The engine assists by:

Problem Discovery

Question Generation

Tool Discovery

Resource Discovery

Evidence Discovery

Human creates novelty.

AI assists discovery.

24.13 Create New Workflow

Problem Description

↓

Evidence Discovery

↓

Question Generation

↓

Tool Discovery

↓

Resource Discovery

↓

Recommendation

↓

Human Creation

24.14 Tool Recommendation Model

Tools may be recommended only if evidence exists.

Supported Tool Evidence

Official Documentation

Official Usage Statistics

Official Provider Information

Official Resource Pages

AI must not invent tool claims.

24.15 Recommendation Evidence Model

Every recommendation must contain:

Evidence

Citation

Source

Timestamp

Ranking Metadata

Audit Metadata

Mandatory fields.

24.16 Recommendation Audit Model

Every recommendation must support:

Recommendation Audit

Evidence Audit

Citation Audit

Source Audit

Ranking Audit

Audit records remain immutable.

24.17 Recommendation Traceability Model

Every recommendation must answer:

Why was this recommended?

What evidence supports it?

What citations support it?

What source supports it?

How was it ranked?

24.18 Recommendation Governance Rules

The platform must continuously answer:

Can every recommendation be explained?

Can every recommendation be audited?

Can every recommendation be reproduced?

Can every recommendation be traced?

These rules are mandatory.

24.19 Recommendation Scalability Rules

The Recommendation Engine must support:

Millions of Recommendations

Millions of Evidence Links

Millions of Ranking Records

Historical Recommendation Versions

without redesigning the architecture.

Recommended V1 Stack

FastAPI

PostgreSQL

pgvector

BM25

Hybrid Retrieval

Ranking Engine

Audit Engine

Compatible with zero-cost MVP deployment.

24.20 Section 24 Acceptance Criteria

Section 24 is complete only if:

- ✓ Discovery recommendations are supported
- ✓ Online recommendations are supported
- ✓ Physical recommendations are supported
- ✓ Create Similar recommendations are supported
- ✓ Create New recommendations are supported
- ✓ Ranking integration is supported
- ✓ Evidence linkage is supported
- ✓ Auditability is supported
- ✓ Traceability is supported
- ✓ Enterprise-scale recommendation generation is supported

DDD V1 – Section 25

Deployment & Infrastructure Domain Architecture

25.1 Purpose

The Deployment & Infrastructure Domain defines how TODP V1 is deployed, operated, monitored and scaled.

The purpose of this domain is to ensure:

- Reliability
- Availability
- Traceability
- Auditability
- Reproducibility
- Cost Efficiency

while remaining compatible with the V1 zero-cost MVP strategy.

25.2 Infrastructure Philosophy

TODP follows:

Simple First

Scale Later

and

One Deployable System

One Source Of Truth

and

Infrastructure Must Be Observable

Infrastructure complexity must not exceed product complexity.

25.3 V1 Deployment Architecture

Frontend
↓
API Layer
↓
Agent Layer
↓
Retrieval Layer
↓
Database Layer

Technology Stack

Frontend
=
React

Backend
=
FastAPI

Database
=
PostgreSQL

Vector Search
=
pgvector

Caching
=
Redis

Observability
=
Prometheus + Grafana

25.4 Infrastructure Components

Core Components

Web Application

API Server

Agent Services

PostgreSQL

pgvector

Redis

Monitoring Stack

All components must support audit logging.

25.5 Environment Architecture

Supported Environments

Development

Testing

Staging

Production

Purpose

Support safe deployment workflows.

25.6 Development Environment

Purpose

Developer productivity.

Characteristics

Local Deployment

Single Database

Single API

Mock Integrations

Local Observability

Target

Zero Cost

25.7 Testing Environment

Purpose

Validate:

Retrieval

Ranking

Recommendations

Audits

Agents

Characteristics

Automated Testing

Regression Testing

Evaluation Pipelines

25.8 Staging Environment

Purpose

Production simulation.

Characteristics

Production Configuration

Limited Data

Full Audit Pipelines

25.9 Production Environment

Purpose

Serve real users.

Characteristics

High Availability

Backups

Monitoring

Observability

Disaster Recovery

25.10 Container Architecture

Recommended V1 Deployment

Docker

Container Layout

frontend

backend

postgres

redis

monitoring

Purpose

Portable deployment.

25.11 Database Infrastructure

Primary Database

PostgreSQL

Extensions

pgvector

pg_trgm

uuid-oss

Purpose

Support:

Transactional Data

Vector Search

Hybrid Retrieval

25.12 Retrieval Infrastructure

Supported Retrieval

BM25

Vector Search

Hybrid Retrieval

Reranking

Pipeline

Documents

↓

Chunks

↓

Embeddings

↓

Retrieval

25.13 Agent Infrastructure

Supported Agents

Discovery Agent

Authority Agent

Retrieval Agent

Evidence Agent

Ranking Agent

Recommendation Agent

Registration Agent

Calendar Agent

Audit Agent

Hallucination Audit Agent

Agents may initially run inside:

Single FastAPI Service

25.14 Monitoring Infrastructure

Recommended Stack

OpenTelemetry

Prometheus

Grafana

Monitor

API Performance

Database Performance

Retrieval Performance

Recommendation Performance

Agent Performance

25.15 Logging Infrastructure

Every service must log:

Request ID

Trace ID

Timestamp

Agent ID

Audit ID

Purpose

Support:

- Traceability
 - Auditability
 - Reproducibility
-

25.16 Backup Architecture

Backup Scope

Database

Audit Records

Evidence Records

Recommendation Records

Configuration

Recommended Schedule

Daily Backup

Weekly Snapshot

25.17 Disaster Recovery Model

Recovery Targets

Database Recovery

Configuration Recovery

Audit Recovery

Evidence Recovery

Purpose

Maintain platform continuity.

25.18 Infrastructure Governance Rules

The platform must continuously answer:

Can the system be recovered?

Can deployments be reproduced?

Can failures be traced?

Can services be monitored?

These rules are mandatory.

25.19 Infrastructure Scalability Roadmap

V1

Single Server

PostgreSQL

pgvector

FastAPI

V2

Multiple Services

Dedicated Retrieval Layer

Dedicated Agent Layer

V3

Distributed Infrastructure

Distributed Retrieval

Distributed Agents

Multi-Region Deployment

25.20 Section 25 Acceptance Criteria

Section 25 is complete only if:

- ✓ Development environment is defined
- ✓ Testing environment is defined
- ✓ Staging environment is defined
- ✓ Production environment is defined
- ✓ Docker deployment is supported
- ✓ PostgreSQL infrastructure is supported
- ✓ pgvector infrastructure is supported
- ✓ Agent infrastructure is supported

- ✓ Monitoring infrastructure is supported
 - ✓ Disaster recovery strategy is defined
-

DDD V1 – Section 26

Analytics Domain Architecture

26.1 Purpose

The Analytics Domain is responsible for measuring, storing and analyzing platform activity across TODP.

The purpose of this domain is to understand:

- Discovery Effectiveness
- Recommendation Effectiveness
- Registration Effectiveness
- Retrieval Effectiveness
- System Usage
- Platform Growth

while maintaining TODP's Evidence-First philosophy.

Analytics is used for measurement.

Analytics is not used for recommendation manipulation.

26.2 Analytics Philosophy

TODP follows:

Measure Everything

Manipulate Nothing

and

Analytics
≠
Ranking

and

Analytics
≠
Recommendation

V1 recommendations remain driven by:

Money Cost

Time Cost

Distance Cost

Analytics never overrides ranking.

26.3 Analytics Domain Entity Diagram

```
analytics_events
├── discovery_analytics
├── recommendation_analytics
├── registration_analytics
├── retrieval_analytics
├── usage_analytics
├── analytics_dashboards
└── analytics_reports
```

26.4 Analytics Events Table

Primary Entity

analytics_events

Purpose

Store platform activity.

Primary Key

event_id

Core Fields

event_id

event_type

entity_type

entity_id

event_timestamp

created_at

Examples

Search Performed

Recommendation Viewed

Registration Started

Registration Completed

Calendar Event Created

26.5 Discovery Analytics Table

Primary Entity

discovery_analytics

Purpose

Measure discovery behavior.

Core Fields

discovery_metric_id

query_id

result_count

discovery_scope

created_at

Examples

Scholarship Search

Program Search

Event Search

Course Search

26.6 Recommendation Analytics Table

Primary Entity

recommendation_analytics

Purpose

Measure recommendation activity.

Core Fields

recommendation_metric_id

recommendation_id

view_count

interaction_count

created_at

Measured Metrics

Views

Clicks

Evidence Reviews

26.7 Registration Analytics Table

Primary Entity

registration_analytics

Purpose

Measure registration outcomes.

Core Fields

registration_metric_id

registration_id

completion_status

completion_time

created_at

Examples

Started

Completed

Abandoned

26.8 Retrieval Analytics Table

Primary Entity

retrieval_analytics

Purpose

Measure retrieval performance.

Core Fields

retrieval_metric_id
retrieval_run_id
retrieval_latency
retrieval_count
created_at

Measured Metrics

BM25 Performance
Vector Performance
Hybrid Performance
Reranking Performance

26.9 Usage Analytics Table

Primary Entity

usage_analytics

Purpose

Measure platform adoption.

Core Fields

usage_metric_id
user_id
session_count
activity_count
created_at

Examples

Daily Active Users

Weekly Active Users

Monthly Active Users

26.10 Analytics Dashboard Table

Primary Entity

analytics_dashboards

Purpose

Store dashboard definitions.

Examples

Discovery Dashboard

Recommendation Dashboard

Registration Dashboard

Retrieval Dashboard

Growth Dashboard

26.11 Analytics Reports Table

Primary Entity

analytics_reports

Purpose

Store generated reports.

Core Fields

report_id
report_name
report_type
generated_at
version

26.12 Discovery Effectiveness Model

The platform measures:

Searches Performed
Results Returned
Evidence Reviews
Registrations Started

Purpose

Understand discovery quality.

26.13 Recommendation Effectiveness Model

The platform measures:

Recommendations Viewed
Recommendations Opened
Evidence Reviews
Registration Conversions

Purpose

Measure usefulness.

26.14 Registration Effectiveness Model

The platform measures:

Registration Starts

Registration Completions

Registration Abandonment

Completion Time

Purpose

Measure actionability.

26.15 Create Similar Analytics

The platform measures:

Opportunities Reviewed

Tools Viewed

Resources Viewed

Creation Sessions Started

Purpose

Understand creation assistance usage.

26.16 Create New Analytics

The platform measures:

Problems Submitted

Tools Reviewed

Resources Reviewed

Artifacts Created

Purpose

Measure innovation support.

26.17 Analytics Governance Rules

Analytics may be used for:

Measurement

Monitoring

Reporting

Product Improvement

Analytics may not be used for:

Sponsored Ranking

Paid Placement

Behavior Manipulation

Hidden Recommendation Bias

26.18 Analytics Audit Model

Every analytics record must support:

Analytics Audit

Data Audit

Metric Audit

Report Audit

Audit records remain immutable.

26.19 Analytics Scalability Rules

The Analytics Domain must support:

Billions of Events

Millions of Reports

Millions of Dashboards

Long-Term Historical Storage

without redesigning the architecture.

Recommended V1 Stack

PostgreSQL

Grafana

Metabase

Prometheus

All are open-source and compatible with zero-cost deployment.

26.20 Section 26 Acceptance Criteria

Section 26 is complete only if:

- ✓ Analytics events are supported
- ✓ Discovery analytics are supported
- ✓ Recommendation analytics are supported
- ✓ Registration analytics are supported
- ✓ Retrieval analytics are supported

- ✓ Usage analytics are supported
 - ✓ Dashboarding is supported
 - ✓ Reporting integration is supported
 - ✓ Auditability is supported
 - ✓ Enterprise-scale analytics is supported
-

DDD V1 – Section 27

Reporting Domain Architecture

27.1 Purpose

The Reporting Domain is responsible for generating, storing and distributing reports across TODP.

The purpose of reporting is to provide:

- Operational Visibility
- Recommendation Transparency
- Audit Transparency
- Analytics Visibility
- Governance Visibility
- Platform Performance Visibility

Reports must be evidence-backed and auditable.

27.2 Reporting Philosophy

TODP follows:

Report What Happened

Not What Is Assumed

and

Every Report

Must Be Traceable

and

Every Report

Must Be Auditable

Reports summarize evidence.

Reports do not create evidence.

27.3 Reporting Domain Entity Diagram

```
reports
├── report_templates
├── report_runs
├── report_exports
├── report_schedules
├── report_distributions
├── analytics_reports
└── audit_logs
```

27.4 Reports Table

Primary Entity

reports

Purpose

Store generated reports.

Primary Key

report_id

Core Fields

report_id
report_name
report_type
report_status
generated_at
version

Examples

Recommendation Report
Audit Report
Analytics Report
Registration Report
System Report

27.5 Report Templates Table

Primary Entity

report_templates

Purpose

Store reusable report definitions.

Core Fields

template_id
template_name
template_type

created_at

version

Examples

Weekly Audit Report

Monthly Analytics Report

Recommendation Quality Report

System Health Report

27.6 Report Runs Table

Primary Entity

report_runs

Purpose

Track report generation.

Core Fields

report_run_id

template_id

generation_start

generation_end

status

created_at

Supported Status

Running

Completed

Failed

Cancelled

27.7 Report Exports Table

Primary Entity

`report_exports`

Purpose

Track exported reports.

Core Fields

`export_id`

`report_id`

`export_format`

`export_timestamp`

`created_at`

Supported Formats

PDF

DOCX

CSV

XLSX

JSON

27.8 Report Schedules Table

Primary Entity

`report_schedules`

Purpose

Store report scheduling rules.

Core Fields

`schedule_id`
`report_id`
`schedule_type`
`next_execution`
`created_at`

Examples

Daily
Weekly
Monthly
Quarterly

27.9 Report Distribution Table

Primary Entity

`report_distributions`

Purpose

Store report delivery information.

Core Fields

`distribution_id`
`report_id`
`recipient_type`

distribution_method

created_at

Examples

Dashboard

Email

API

Export Download

27.10 Recommendation Reports

Purpose

Provide visibility into:

Recommendations Generated

Recommendations Audited

Recommendation Failures

Recommendation Coverage

27.11 Audit Reports

Purpose

Provide visibility into:

Audit Pass Rate

Audit Failures

Citation Failures

Evidence Failures

Hallucination Findings

27.12 Analytics Reports

Purpose

Provide visibility into:

Discovery Metrics

Recommendation Metrics

Registration Metrics

Usage Metrics

27.13 Registration Reports

Purpose

Provide visibility into:

Registrations Started

Registrations Completed

Registrations Abandoned

Completion Rates

27.14 Authority Reports

Purpose

Provide visibility into:

Authorities Discovered

Authorities Validated

Hierarchy Growth

Authority Coverage

27.15 Retrieval Reports

Purpose

Provide visibility into:

BM25 Performance

Vector Performance

Hybrid Performance

Retrieval Quality

27.16 Operational Reports

Purpose

Provide visibility into:

System Health

API Health

Database Health

Agent Health

Infrastructure Health

27.17 Reporting Governance Rules

The platform must continuously answer:

Can every report be reproduced?

Can every report be audited?

Can every report be traced?

Can every report be explained?

These rules are mandatory.

27.18 Reporting Audit Model

Every report must support:

Source Audit

Evidence Audit

Report Audit

Generation Audit

Audit records remain immutable.

27.19 Reporting Scalability Rules

The Reporting Domain must support:

Millions of Reports

Millions of Exports

Millions of Scheduled Executions

Long-Term Historical Storage

without redesigning the architecture.

Recommended V1 Stack

Metabase

Grafana

PostgreSQL

FastAPI Reporting APIs

All are open-source and compatible with zero-cost deployment.

27.20 Section 27 Acceptance Criteria

Section 27 is complete only if:

- ✓ Report generation is supported
 - ✓ Report templates are supported
 - ✓ Report scheduling is supported
 - ✓ Report exports are supported
 - ✓ Report distribution is supported
 - ✓ Recommendation reporting is supported
 - ✓ Audit reporting is supported
 - ✓ Analytics reporting is supported
 - ✓ Operational reporting is supported
 - ✓ Enterprise-scale reporting is supported
-

DDD V1 – Section 28

Cost Optimization Domain Architecture

28.1 Purpose

The Cost Optimization Domain is responsible for minimizing the operational cost of TODP while preserving:

- Traceability
- Auditability
- Reproducibility
- Recommendation Quality
- Retrieval Quality

The purpose of this domain is to ensure TODP can operate efficiently from:

Zero-Cost MVP
↓
Early Startup
↓
Growth Stage
↓
Enterprise Scale

without unnecessary infrastructure spending.

28.2 Cost Optimization Philosophy

TODP follows:

Minimize Cost

Maximize Evidence

and

Optimize Infrastructure

Not Trust

and

Evidence Quality

Must Not Be Reduced

for cost reasons.

28.3 Cost Optimization Scope

This domain manages:

Infrastructure Cost

Storage Cost

Retrieval Cost

Embedding Cost

LLM Cost

Monitoring Cost

Reporting Cost

28.4 Cost Architecture Overview

User

↓

Retrieval

↓

Evidence

↓

Ranking

↓

Recommendation

Cost optimization is applied to:

Infrastructure

Storage

Processing

Generation

but never to evidence integrity.

28.5 Infrastructure Cost Model

Primary Cost Sources

Compute

Database

Storage

Network

Monitoring

V1 Goal

Single Server Deployment

Target

Lowest Possible Operational Cost

28.6 Database Cost Optimization

V1 Database

PostgreSQL
+
pgvector

Optimization Rules

Single Database

Shared Infrastructure

No Dedicated Vector DB

No Distributed Database

Purpose

Minimize infrastructure complexity.

28.7 Retrieval Cost Optimization

TODP supports:

BM25

Vector Search

Hybrid Retrieval

Retrieval Strategy

Metadata Filters

↓
BM25
↓
Vector Search
↓
Reranking

Purpose

Reduce expensive retrieval operations.

28.8 Embedding Cost Optimization

Embeddings are generated:

During Ingestion

not during recommendation generation.

Workflow

Document
↓
Chunk
↓
Embedding
↓
Storage

Purpose

Avoid repeated embedding costs.

28.9 LLM Cost Optimization

LLMs are used for:

Summarization

Explanation

Create Similar

Create New

LLMs are NOT used for:

Ranking

Evidence Storage

Audit Storage

Citation Storage

Purpose

Reduce token costs.

28.10 Recommendation Cost Optimization

Recommendation Pipeline

Retrieval

↓

Evidence

↓

Ranking

↓

Recommendation

Rule

Recommendations should reuse:

Existing Evidence

Existing Citations

Existing Sources

Avoid repeated processing.

28.11 Monitoring Cost Optimization

Recommended Stack

OpenTelemetry

Prometheus

Grafana

Purpose

Provide observability using open-source tools.

28.12 Storage Cost Optimization

Priority Levels

Hot Data

Warm Data

Cold Data

Archive Data

Examples

Active Recommendations → Hot

Audit Logs → Warm

Historical Reports → Cold

Archived Snapshots → Archive

28.13 Caching Architecture

Recommended Cache

Redis

Cache Candidates

Search Results

Recommendation Results

Authority Lookups

Metadata Filters

Purpose

Reduce repeated computation.

28.14 Agent Cost Optimization

V1 Agent Deployment

Single FastAPI Process

Purpose

Avoid:

Microservice Overhead

Container Sprawl

Infrastructure Duplication

28.15 Discovery Cost Optimization

TODP Discovery Hierarchy

Institution
↓
Local
↓
District
↓
State
↓
National
↓
International

Purpose

Reduce:

Money Cost

Time Cost

Distance Cost

for users.

28.16 User Cost Optimization Model

Every recommendation should attempt to minimize:

Money Cost

Time Cost

Distance Cost

Online opportunities:

Distance Cost = 0

Travel Time = 0

This principle aligns recommendation cost with user value.

28.17 Cost Governance Rules

The platform must continuously answer:

Where is money being spent?

Which component costs the most?

Can costs be reduced?

Can optimization be measured?

These rules are mandatory.

28.18 Cost Audit Model

Every cost decision must support:

Infrastructure Audit

Storage Audit

Retrieval Audit

LLM Usage Audit

Optimization Audit

Audit records remain immutable.

28.19 Cost Scalability Roadmap

V1

Single Server

PostgreSQL

pgvector

Redis

V2

Dedicated Retrieval Layer

Dedicated Agent Layer

Separate Monitoring Stack

V3

Distributed Infrastructure

Distributed Retrieval

Multi-Region Deployment

Cost increases only when justified by usage.

28.20 Section 28 Acceptance Criteria

Section 28 is complete only if:

- ✓ Infrastructure cost model is defined
- ✓ Database cost model is defined
- ✓ Retrieval cost model is defined
- ✓ Embedding cost model is defined
- ✓ LLM cost model is defined
- ✓ Storage optimization is defined
- ✓ Caching strategy is defined
- ✓ User cost optimization is defined

- ✓ Cost auditing is supported
 - ✓ Zero-cost MVP path is supported
-

DDD V1 – Section 29

V2 / V3 Architecture Roadmap

29.1 Purpose

This section defines the future evolution of TODP beyond V1.

The purpose is to provide a controlled roadmap from:

V1
↓
V2
↓
V3

while preserving:

- Evidence First Architecture
 - Traceability
 - Auditability
 - Reproducibility
 - Cost-Aware Recommendations
-

29.2 Roadmap Philosophy

TODP follows:

Build What Can Be Verified First

and

Expand Only After Auditability Exists

and

Complexity Must Earn Its Place

V1 establishes the foundation.

V2 expands intelligence.

V3 expands ecosystem scale.

29.3 Version Overview

V1

=

Evidence Discovery Platform

V2

=

Opportunity Creation Platform

V3

=

Global Opportunity Ecosystem

29.4 V1 Scope (Frozen)

V1 includes:

Discovery

Retrieval

Evidence

Citations

Recommendations

Registration Assistance

Calendar Assistance

Auditability

Hallucination Detection

Ranking Formula

Money Cost
+
Time Cost
+
Distance Cost

No sponsored content.

No paid placement.

No recommendation manipulation.

29.5 V2 Vision

Purpose

Transform TODP from:

Opportunity Discovery

into:

Opportunity Creation

New Capabilities

Creation Workspaces

Community Projects

Collaboration

Resource Sharing

Project Tracking

29.6 V2 Creation Engine

Expansion of:

Create Something Similar

Create Something New

New Workflow

Problem

↓

Evidence

↓

Tools

↓

Resources

↓

Plan

↓

Execution

↓

Artifact

Purpose

Support creation beyond discovery.

29.7 V2 Artifact Registry

New Domain

artifact_registry

Purpose

Store:

Projects

Research

Events

Workshops

Programs

Community Initiatives

created by users.

29.8 V2 Contribution Model

Every created artifact stores:

Creator

Evidence Used

Tools Used

Resources Used

Creation Timestamp

Purpose

Preserve provenance.

29.9 V2 Popularity Metrics

V1 Ranking remains unchanged.

V2 introduces observable metrics:

Views

Registrations

Downloads

Bookmarks

Shares

Rule

Popularity does not replace evidence.

Popularity becomes additional metadata.

29.10 V2 Community Layer

Supports:

Communities

Teams

Project Groups

Learning Groups

Event Organizers

Purpose

Enable collaborative creation.

29.11 V2 Ecosystem Registry

New Domain

`ecosystem_registry`

Purpose

Store relationships between:

Projects

Events

Programs

Communities

Resources

29.12 V2 Recommendation Enhancements

Additional Inputs

Artifact Usage

Community Activity

Completion Statistics

Rule

Evidence remains mandatory.

29.13 V3 Vision

Purpose

Transform TODP into:

Global Opportunity Infrastructure

Scope

Institutions

Governments

Universities

Organizations

Communities

Individuals

29.14 V3 Global Authority Network

Expansion of:

Authority Registry

Supports:

Global Authorities

Cross-Country Authorities

International Programs

Multi-Level Jurisdictions

29.15 V3 Knowledge Graph

New Architecture

Authority
↔ Opportunity
↔ Evidence
↔ Citation
↔ Source
↔ Artifact

Purpose

Support advanced discovery.

29.16 V3 Multi-Agent Ecosystem

Expansion of:

Discovery Agents

Evidence Agents

Recommendation Agents

into:

Specialized Domain Agents

Examples

Education Agent

Scholarship Agent

Research Agent

Startup Agent

Government Program Agent

29.17 V3 Infrastructure Evolution

V1

Single Server

V2

Distributed Services

V3

Distributed Global Platform

Supports:

Multi-Region Deployment

Global Availability

Regional Data Layers

29.18 V3 Revenue Evolution

V1

No Sponsored Content

V2

Subscription Services

V3

Enterprise Integrations

Institutional Subscriptions

Platform Services

Rule

Sponsored ranking remains prohibited.

29.19 Roadmap Governance Rules

Every future version must preserve:

Evidence First

Traceability

Auditability

Reproducibility

Authority Validation

These principles are immutable.

29.20 Section 29 Acceptance Criteria

Section 29 is complete only if:

✓ V1 roadmap is defined

✓ V2 roadmap is defined

- ✓ V3 roadmap is defined
 - ✓ Creation ecosystem is defined
 - ✓ Artifact registry is defined
 - ✓ Community layer is defined
 - ✓ Global authority network is defined
 - ✓ Knowledge graph evolution is defined
 - ✓ Revenue evolution is defined
 - ✓ Governance continuity is defined
-

DDD V1 – Section 30

Final System Architecture Blueprint

30.1 Purpose

This section defines the complete architecture blueprint for TODP V1.

The blueprint consolidates all previously defined domains into a single coherent system architecture.

This blueprint serves as the final architectural reference for:

- Development
 - Deployment
 - Auditing
 - Scaling
 - Future Evolution
-

30.2 System Vision

TODP V1 is an:

Evidence-First

Opportunity Discovery Platform

that enables users to:

Discover Opportunities

Create Similar Opportunities

Create New Opportunities

using:

Evidence

Citations

Sources

Authority Validation

Auditability

Traceability

30.3 Core Product Journeys

TODP V1 supports four primary journeys.

Journey 1

Discover Online

Flow

Search

↓

Retrieval

↓

Evidence

↓

Ranking
↓
Recommendation
↓
Evidence Review
↓
Registration
↓
Calendar

Journey 2

Discover Physical

Flow

Search
↓
Location Discovery
↓
Authority Discovery
↓
Evidence
↓
Ranking
↓
Recommendation
↓
Registration
↓
Calendar

Journey 3

Create Something Similar

Flow

Existing Opportunity
↓
Evidence Discovery
↓
Tool Discovery
↓
Resource Discovery
↓
Recommendation
↓
Human Action

Journey 4

Create Something New

Flow

Problem
↓
Evidence Discovery
↓
Question Generation
↓
Tool Discovery
↓
Resource Discovery
↓
Recommendation
↓
Human Creation

30.4 System Architecture Overview

User
↓
Frontend
↓
API Layer
↓
Agent Layer
↓
Retrieval Layer
↓
Evidence Layer
↓
Ranking Layer
↓
Recommendation Layer
↓
Audit Layer
↓
Database Layer

30.5 Domain Architecture Overview

TODP V1 consists of:

User Domain

Authority Domain

Opportunity Domain
Evidence Domain
Source Domain
Citation Domain
Recommendation Domain
Ranking Domain
Audit Domain
Hallucination Audit Domain
Retrieval Domain
Document Domain
Vector Search Domain
Registration Domain
Calendar Domain
Agent Domain
Observability Domain
API Domain
Security Domain
Data Governance Domain
Authority Discovery Domain
Recommendation Engine Domain
Deployment Domain
Analytics Domain
Reporting Domain
Cost Optimization Domain

30.6 Discovery Hierarchy Blueprint

TODP V1 Discovery Hierarchy

Institution

↓
Local
↓
District
↓
State
↓
National
↓
International

Purpose

Minimize Money Cost

Minimize Time Cost

Minimize Distance Cost

before expanding outward.

30.7 Universal Ranking Blueprint

All recommendation paths use:

Ranking
↓
Recommendation

V1 Ranking Formula

Total Cost

=

Money Cost
+
Time Cost
+
Distance Cost

Rule

Lowest Total Cost

Ranks Highest

30.8 Authority Blueprint

Recommendations originate from:

Validated Authorities

Examples

Governments

Universities

Departments

Organizations

Foundations

Program Providers

Rule

AI Is Not An Authority

30.9 Retrieval Blueprint

Supported Retrieval

BM25

Vector Search

Hybrid Retrieval

Reranking

Pipeline

Query

↓

Retrieval

↓

Evidence Candidates

30.10 Evidence Blueprint

Every recommendation must contain:

Evidence
Citation
Source
Timestamp
Ranking Metadata
Audit Metadata

Mandatory.

30.11 Audit Blueprint

Every operation must support:

Traceability
Auditability
Reproducibility

Supported Audits

Recommendation Audit
Evidence Audit
Citation Audit
Source Audit
Ranking Audit
Security Audit

30.12 Hallucination Prevention Blueprint

The system continuously validates:

Claims

Evidence

Sources

Citations

Rule

Unsupported Output

=

Flagged Output

30.13 Registration Blueprint

TODP supports:

Registration Assistance

Allowed

Auto Fill

Requirement Discovery

Document Assistance

Not Allowed

Auto Submit

User approval is mandatory.

30.14 Calendar Blueprint

TODP supports:

Availability Checking

Scheduling Assistance

Reminders

Rule

User Consent Required

30.15 Agent Blueprint

TODP V1 Agents

Discovery Agent

Authority Agent

Retrieval Agent

Evidence Agent

Ranking Agent

Recommendation Agent

Citation Agent

Registration Agent

Calendar Agent

Audit Agent

Hallucination Audit Agent

Rule

Every agent execution is auditable.

30.16 Security Blueprint

Core Controls

Authentication

Authorization

Encryption

Audit Logging

Access Model

RBAC

Every access must be traceable.

30.17 Observability Blueprint

Monitor

Infrastructure

Agents

Retrieval

Recommendations

Audits

Hallucinations

Recommended Stack

OpenTelemetry

Prometheus

Grafana

30.18 Infrastructure Blueprint

V1 Deployment

React

FastAPI

PostgreSQL

pgvector

Redis

Docker

Architecture

Single Server

Single Database

Single Deployment

Purpose

Zero-Cost MVP

30.19 Immutable System Principles

TODP must always preserve:

Evidence First

Authority Before Recommendation

Ranking Before Recommendation

Traceability

Auditability

Reproducibility

No Sponsored Rankings

No Paid Placement

These principles remain unchanged across:

V1

V2

V3

30.20 Final Acceptance Criteria

TODP V1 Architecture is complete only if:

- ✓ Discovery is supported
- ✓ Retrieval is supported
- ✓ Evidence is supported
- ✓ Citations are supported
- ✓ Authority validation is supported
- ✓ Ranking is supported
- ✓ Recommendations are supported
- ✓ Registration assistance is supported
- ✓ Calendar assistance is supported
- ✓ Auditability is supported
- ✓ Hallucination detection is supported
- ✓ Security is supported
- ✓ Analytics is supported
- ✓ Reporting is supported

- ✓ Cost optimization is supported
 - ✓ Observability is supported
 - ✓ Scalability roadmap is defined
 - ✓ V2 roadmap is defined
 - ✓ V3 roadmap is defined
 - ✓ All recommendations remain evidence-backed
-